

DEPARTMENT OF DATA SCIENCE AND
ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

A Digital-Twin Simulation Framework for Joint Inventory and Transport Optimisation in Multi-Echelon Supply Chains

A Project Report

Submitted by

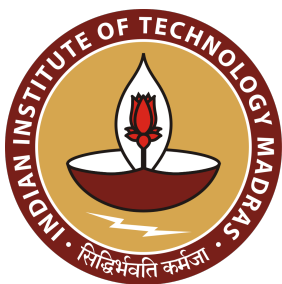
SAKTHE BALAN G K

For the award of the degree

Of

INTERDISCIPLINARY DUAL DEGREE PROGRAMME

May 2026



DEPARTMENT OF DATA SCIENCE AND
ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

A Digital-Twin Simulation Framework for Joint Inventory and Transport Optimisation in Multi-Echelon Supply Chains

A Project Report

Submitted by

SAKTHE BALAN G K

For the award of the degree

Of

INTERDISCIPLINARY DUAL DEGREE PROGRAMME

May 2026

THESIS CERTIFICATE

This is to certify that the Project Report titled **A DIGITAL-TWIN SIMULATION FRAMEWORK FOR JOINT INVENTORY AND TRANSPORT OPTIMISATION IN MULTI-ECHELON SUPPLY CHAINS**, submitted by me to the Indian Institute of Technology Madras, for the award of **Interdisciplinary Dual Degree Programme**, is a bona fide record of the research work done by me under the supervision of **Dr. Sivaraman Ambikasaran and Dr. Sri Vallabha Deevi**. The contents of this Project Report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Chennai 600036

Date: May 2026

Sakthe Balan G K

Dr. Sivaraman Ambikasaran

Associate Professor

Research advisor

Department of Data Science and Artificial Intelligence

IIT Madras

Dr. Sri Vallabha Deevi

Director, Data Science, Tiger Analytics

Adjunct Faculty

Research co-advisor

Department of Data Science and Artificial Intelligence

IIT Madras

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my thesis guides for their invaluable guidance, encouragement, and support throughout the course of this Interdisciplinary Dual Degree Project. Their insights into supply-chain optimisation and simulation methodology shaped the direction of this work in fundamental ways.

I am grateful to the Department of Data Science and Artificial Intelligence, IIT Madras, for providing the computational resources and academic environment that made this project possible.

I would also like to thank the open-source communities behind `SIMPY`, `OPTUNA`, `PYTEST`, and the broader Python scientific ecosystem, without which this framework could not have been built in the time available.

Finally, I thank my family and friends for their patience and encouragement during the long hours this project demanded.

ABSTRACT

KEYWORDS Multi-echelon inventory optimisation; discrete-event simulation; digital twin; transport consolidation; Bayesian optimisation; TPE; bullwhip effect; supply chain; joint optimisation; Pareto frontier

Classical multi-echelon inventory models optimise stock levels in isolation, treating transport as a linear per-unit cost. In practice, vehicle-based transport has a step-function cost structure: a half-full truck costs nearly as much to run as a full one. When transport dominates supply-chain cost, this mismatch makes inventory-only optimisation fundamentally suboptimal.

This project develops a modular, open-source **digital-twin simulation framework** for Multi-Echelon Inventory Optimisation (MEIO) that jointly optimises inventory and transport decisions. The framework implements seven inventory control policies, an arbitrary DAG network topology, and a transport layer that enforces simultaneous volume and weight capacity constraints. A Tree-structured Parzen Estimator (TPE) optimiser searches a mixed-integer space of stock levels and dispatch thresholds; a two-objective NSGA-II mode maps the full cost–service Pareto frontier.

Applied to a 5-SKU, 1-supplier–2-warehouse–3-retailer network driven by real M5 Walmart demand, joint optimisation achieves a 37% total cost reduction over the analytical Clark-Scarf baseline while simultaneously raising fill rate above the 92% service-level target — results that neither inventory-only nor transport-only optimisation can match. In a transport-cost-dominated network, extra warehouse safety stock enables vehicle consolidation and generates substantial savings in transport and shortage costs.

Extension experiments confirm the breadth of the result. The elevated safety stock incidentally absorbs a two-week supply disruption better than the analytical baseline (E5), and policy comparison on equal transport footing shows base-stock remains most cost-effective (E6). The parameters are robust to forecast error (E7), and the per-echelon bullwhip metric confirms negligible amplification under base-stock with deterministic lead times (E8). Stochastic testing (E9) confirms the parameters remain effective under Poisson demand. The framework is validated by 84 unit and integration tests and four preliminary experiments (EV1–EV4), and is fully reproducible.

CONTENTS

	Page
ACKNOWLEDGEMENTS	i
ABSTRACT	iii
LIST OF TABLES	ix
LIST OF FIGURES	xi
CHAPTER 1 INTRODUCTION	1
1.1 Multi-Echelon Inventory Optimisation	1
1.2 Need for a Digital Twin	2
1.3 Contributions	3
1.4 Thesis Structure	5
CHAPTER 2 LITERATURE REVIEW	7
2.1 Single-Echelon Foundations	7
2.1.1 Newsvendor model	7
2.1.2 Base-stock policy	7
2.1.3 (s, S) and (R, Q) policies	8
2.1.4 Economic Order Quantity (EOQ)	8
2.2 Multi-Echelon Optimality — Clark and Scarf (1960)	8
2.3 Periodic-Review and Cyclic Policies	9
2.3.1 (R, S) periodic-review policy	9
2.3.2 (k, m) -cycle policies	9
2.4 The Bullwhip Effect	10
2.5 Transport Consolidation in Inventory Models	10
2.6 Simulation-Optimisation Methods	11
2.6.1 Classical gradient-free approaches	11
2.6.2 Tree-structured Parzen Estimator (TPE)	11
2.6.3 Multi-objective optimisation: NSGA-II	12
2.6.4 Neural and reinforcement-learning approaches	12
2.7 Existing MEIO Software	13
CHAPTER 3 DIGITAL-TWIN ARCHITECTURE	15
3.1 Network Topology	15
3.2 Discrete-Event Simulation Loop	16
3.3 Node State Model	17
3.3.1 Helper methods	18
3.4 Transport Layer	19
3.4.1 TransportOption	19

3.4.2	GreedyLoadPlanner	20
3.4.3	Pluggable protocols	21
3.5	Cost Model	22
3.6	Configuration and Input Methods	23
3.6.1	CSV input pipeline	24
3.7	Supply Disruption Modelling	25
CHAPTER 4 INVENTORY POLICIES		27
4.1	Base-Stock Policy	27
4.2	Reorder-Point Family: (s, S) and (R, Q)	28
4.2.1	(s, S) policy	28
4.2.2	(R, Q) policy	29
4.3	Periodic-Review: (R, S) and Order-Up-To with Phase Offset	29
4.3.1	(R, S) periodic-review policy	29
4.3.2	Order-up-to with phase offset	29
4.4	Cyclic Policy: (k, m) -Cycle Scheduling	30
4.5	Echelon Base-Stock (Clark–Scarf)	30
4.6	Policy Comparison	31
CHAPTER 5 OPTIMISATION FRAMEWORK		33
5.1	Need for Optuna TPE	33
5.2	Search-Space Encoding	34
5.2.1	Inventory: base-stock levels	34
5.2.2	Transport: dispatch thresholds	34
5.3	Joint Search	35
5.4	Constrained Optimisation — Minimum Fill Rate	35
5.4.1	Aggregate vs per-SKU constraint	36
5.5	Pareto Exploration	36
5.5.1	Primary: NSGA-II two-objective study	36
5.5.2	Sanity check: constraint sweep	37
5.6	Re-Evaluation and Search Space Summary	37
5.6.1	Search space dimensionality	37
CHAPTER 6 EXPERIMENTAL METHODOLOGY		39
6.1	Preliminary Validation Experiments (EV1–EV4)	39
6.1.1	EV1 — Constant-demand convergence	39
6.1.2	EV2 — Demand shock response	39
6.1.3	EV3 — Policy comparison under transport capacity	40
6.1.4	EV4 — Capacity scaling	40
6.2	Dataset: M5 Walmart Forecasting Data	41
6.3	Main Experiment Configuration	42
6.4	Experimental Controls	42
6.4.1	Multi-seed statistical validation	43
6.5	Bullwhip Metric	44

6.6	Main Optimisation Experiments E0–E9	44
6.6.1	Newsvendor levels at runtime (E0)	46
6.6.2	Disruption injection (E5)	46
6.7	Reproducibility	47
CHAPTER 7 RESULTS AND DISCUSSION		49
7.1	Preliminary Validation Results (EV1–EV4)	49
7.1.1	EV1 — Constant-demand convergence	49
7.1.2	EV2 — Demand shock propagation	49
7.1.3	EV3 — Policy comparison under transport capacity	50
7.1.4	EV4 — Capacity scaling on M5 data	51
7.2	E0 — Analytical Baseline	52
7.3	E1a — Fixed 25% Dispatch Threshold	53
7.4	E1b — Transport-Only Optimisation	54
7.5	E2 — Inventory-Only Optimisation	54
7.6	E3 — Joint Inventory + Transport Optimisation	55
7.6.1	Mechanism of joint optimisation	55
7.6.2	Per-SKU fill rates	56
7.7	E3_per_sku — Joint Optimisation with Per-SKU Constraint	57
7.8	E4 — Pareto Frontier: Cost vs Service	57
7.8.1	NSGA-II frontier (primary)	57
7.8.2	Constraint sweep (sanity check)	58
7.9	E5 — Disruption Robustness	59
7.10	E6 — Policy Comparison (Core 4)	59
7.11	E7 — Forecast Sensitivity	61
7.12	E8 — Bullwhip-Aware Joint Optimisation	63
7.13	E9 — Stochastic Demand Robustness	64
7.14	Sensitivity to Transport Cost Ratio	64
7.15	Cross-Experiment Comparison	66
7.16	Bullwhip Analysis	67
7.17	Deterministic Reproducibility Check	69
CHAPTER 8 CONCLUSION AND FUTURE WORK		71
8.1	Summary of Contributions	71
8.2	Future Work	73
8.3	Conclusion	75
APPENDIX A CONFIGURATION SCHEMA REFERENCE		77
APPENDIX B EXPERIMENT PARAMETERS AND RUNTIMES		81
APPENDIX C TEST SUITE SUMMARY		85
APPENDIX D REPRODUCING THE RESULTS		87
BIBLIOGRAPHY		91

LIST OF TABLES

Table	Caption	Page
3.1	CSV input files and their roles in the configuration pipeline.	25
4.1	Summary of the seven implemented inventory policies.	31
5.1	Search space dimensionality by optimisation mode (1N3 with $N = 2$ warehouses, 5 SKUs).	38
6.1	Preliminary validation experiments: scope and purpose.	40
6.2	Network topologies and demand configurations used across all experiments.	40
6.3	Deterministic lead times in the canonical 1N3 configuration.	42
6.4	Main experiment simulation parameters.	42
6.5	Main optimisation experiments: design and hypothesis or finding.	45
7.1	EV4 capacity scaling results (single-SKU $2W \times 3R$ topology).	52
7.2	E0 cost decomposition and fill rate.	52
7.3	E1a cost decomposition and fill rate.	53
7.4	E1b cost decomposition and fill rate.	54
7.5	E2 cost decomposition and fill rate.	55
7.6	E3 cost decomposition and fill rate.	55
7.7	Cost composition shift from E0 to E3.	56
7.8	E3 fill rates by SKU.	56
7.9	E3_per_sku cost decomposition and fill rate.	57
7.10	E4 NSGA-II Pareto frontier (primary).	58
7.11	E5 disruption robustness — same 14-day W1 outage, two parameter sets.	59
7.12	E6 policy comparison — each policy jointly optimises inventory parameters and dispatch thresholds.	60
7.13	E7 forecast sensitivity sweep.	62
7.14	E8 bullwhip-aware sweep.	63
7.15	E9 stochastic demand robustness — E3 params evaluated over 10 Poisson seeds.	64
7.16	Transport cost ratio sensitivity: E0 (analytical baseline) and E3 (joint optimisation) at three cost multipliers.	65
7.17	Summary of core optimisation, disruption, and policy-comparison experiments (E7–E9 summarised in their respective tables above).	66
7.18	Per-echelon bullwhip ratios for the canonical experiments (median across SKUs).	68
7.19	Multi-seed determinism check for E3 best parameters (canonical deterministic scenario).	69
B.1	E3 optimised base-stock levels.	81
B.2	E3 optimised dispatch thresholds.	81
B.3	E9 stochastic robustness results (10 Poisson seeds, E3 params).	82

B.4	Per-trial wall-clock time (20-trial sample, averaged).	83
B.5	Per-experiment wall-clock time (<code>--trials 100</code> , single-threaded). . . .	83

LIST OF FIGURES

Figure	Caption	Page
3.1	Discrete-event simulation loop for one daily tick.	17
7.1	EV1 — Inventory position at retailer and warehouse under constant demand.	50
7.2	EV2 — On-hand inventory under a demand spike at day 20.	50
7.3	EV3a — Warehouse inventory under base-stock and (s, S) policies (without transport capacity).	51
7.4	EV3b — Warehouse inventory under base-stock and (s, S) policies (with transport capacity).	51
7.5	E5 — Cost and fill rate under the 14-day W1 outage.	60
7.6	E6 — Cost and fill rate for each individually optimised policy.	61
7.7	E7 — Total cost and fill rate vs. forecast error σ_f	62
7.8	Fill rate across experiments E0–E3_per_sku.	67
7.9	Cost breakdown by category across experiments E0–E3_per_sku.	67

CHAPTER 1

INTRODUCTION

1.1 MULTI-ECHELON INVENTORY OPTIMISATION

Inventory management decisions lie at the heart of supply-chain efficiency, resilience, and cost-effectiveness. Modern supply chains are hierarchical: goods flow from manufacturing centres through distribution centres (DCs) to retail outlets, with inventory held at each stage. Managing inventory across these stages — a problem known as **Multi-Echelon Inventory Optimisation (MEIO)** — requires making critical replenishment decisions under demand uncertainty, transportation limitations, and operational constraints, often without clear visibility into system-wide consequences.

Three compounding difficulties make the MEIO problem harder than its single-echelon counterpart:

1. **Coupling of stockouts.** A stockout at a DC propagates downstream. retailers awaiting replenishment accumulate unfilled demand (backlog), incurring shortage costs. Simultaneously, the DC's depleted inventory triggers a large replenishment order upstream, stressing the supplier's capacity.
2. **Lead-time amplification.** The effective replenishment lead time seen by a retailer is the cumulative sum of transit times across all upstream echelons. With a 2-day supplier-to-DC lead time and a 1-day DC-to-retailer lead time, the retailer's effective lead time is 3 days — and any safety stock calculation must account for demand variability over the entire 3-day window, not just the final mile.
3. **The bullwhip effect.** Small fluctuations in end-customer demand are amplified as orders propagate upstream. A retailer reacting to a demand spike places a large order at the DC; the DC, uncertain of future demand, orders even more from the

supplier. This amplification can cause the supplier to swing between feast and famine, eroding efficiency across the chain (Lee *et al.*, 1997).

These interdependencies mean that optimising inventory at each node independently — the standard single-echelon approach — is suboptimal or even infeasible. A system-level view is required.

Compounding this challenge, modifying inventory policies in a live supply chain carries significant operational risk. Downstream impacts on service levels, costs, and system stability are difficult to anticipate in advance: a change to replenishment parameters at the warehouse level may look locally optimal yet trigger bullwhip-amplified order swings at the supplier, or deplete retailer safety stock to dangerous levels several weeks later. This makes experimentation on production systems costly and often impractical.

1.2 NEED FOR A DIGITAL TWIN

The classical solution to the MEIO problem is the Clark-Scarf echelon base-stock policy (Clark and Scarf, 1960), which is optimal for serial networks under a set of idealising assumptions:

- Backlog symmetry: unmet demand is fully backordered at a known linear penalty (no lost sales, no emergency orders).
- Infinite supply at the upstream end: the supplier can always meet any order placed.
- Deterministic lead times: every shipment arrives in exactly L days.
- No transport capacity: vehicles can carry any quantity at the same per-unit cost.

Real supply chains routinely violate all four assumptions. Suppliers have capacity limits; lead times vary with congestion; transport costs are non-linear functions of vehicle utilisation. When transport cost dominates the budget — as in the network analysed here,

where transport accounts for 79.5% of total cost at the analytical baseline — ignoring vehicle capacity and consolidation is highly suboptimal.

In practice, decision-makers frequently rely on simulation tools to evaluate inventory policies and operational scenarios before deployment in order to avoid the operational risks described above. However, many available simulation tools are proprietary, domain-specific, or tightly coupled to particular industrial settings, limiting their transparency, extensibility, and suitability for systematic research — Chapter 2 surveys the landscape in detail. To the best of our knowledge, an open, configurable, MEIO-specific simulation framework that handles multi-SKU demand, arbitrary network topologies, and vehicle-capacity-aware transport has not previously been available. Among the tools surveyed in Section 2.7 (SimPy, AnyLogic, simopt, OR-Gym), none combines these capabilities in a single configurable framework.

A **digital-twin simulation** is a computational replica of a supply chain that can evaluate *any* policy under *any* set of assumptions. Unlike closed-form models, it does not need restrictive assumptions to be tractable. Unlike real-world pilots, it can evaluate thousands of policy configurations in hours rather than months, and unlike proprietary tools, it can be inspected, extended, and adapted to new settings. The simulation-optimisation paradigm — using a fast simulator as the inner loop of a black-box optimiser — has emerged as the practical approach for supply-chain design when analytical solutions are unavailable or inapplicable (Fu *et al.*, 2005).

1.3 CONTRIBUTIONS

This study makes the following research contributions:

1. A **modular, tested digital-twin simulation framework for MEIO**. The framework implements seven inventory control policies — base-stock, (s, S) , (R, Q) , (R, S) , order-up-to with phase offset, (k, m) -cycle, and echelon base-stock — together

with an arbitrary DAG-topology network model and a transport layer that enforces simultaneous volume and weight capacity constraints with tiered cost schedules. The codebase is validated by 84 pytest tests covering simulator invariants, policy behaviour, transport mechanics, and the per-echelon bullwhip metric.

2. **An Optuna-TPE-based joint optimiser with fill-rate constraints.** The optimiser searches over a mixed-integer space of up to 30 parameters (25 base-stock levels + 5 dispatch thresholds) and enforces a minimum fill-rate target via a soft penalty. Three modes are supported: inventory-only, transport-only, and joint. A per-SKU constraint mode ensures no individual SKU is sacrificed to satisfy an aggregate service-level target.
3. **Empirical demonstration that joint optimisation outperforms single-axis optimisation.** On a 5-SKU 1N3 network (1 supplier $\rightarrow$ 2 warehouses $\rightarrow$ 3 retailers) with M5 Walmart demand, joint optimisation achieves a 37% total cost reduction relative to the analytical newsvendor baseline while simultaneously raising fill rate above the 92% service-level target — results that neither inventory-only nor transport-only optimisation can match.
4. **A two-objective Pareto study of the cost–service trade-off.** An NSGA-II optimiser produces a smooth, non-dominated frontier spanning the full range from cost-minimising to service-maximising configurations, revealing a clear knee in marginal cost per fill-rate point and providing a principled basis for service-level negotiation.
5. **Four extension experiments characterising robustness and scope.** Other experiments cover disruption robustness (E5), policy comparison on equal transport footing (E6), forecast-error sensitivity (E7), and bullwhip-aware joint optimisation (E8) — together establishing the generality of the joint optimisation approach.
6. **Stochastic demand robustness and preliminary validation.** The jointly-optimised parameters are shown to transfer to Poisson demand without re-optimisation (E9), and four preliminary validation experiments (EV1–EV4)

confirm simulator correctness against known theoretical behaviours.

1.4 THESIS STRUCTURE

The remainder of this thesis is organised as follows.

Chapter 2 reviews the theoretical foundations: single-echelon newsvendor theory, Clark-Scarf multi-echelon optimality, periodic-review policies, the bullwhip effect, transport consolidation models, and simulation-optimisation methods.

Chapter 3 describes the digital-twin framework architecture: the DAG network model, the discrete-event simulation loop, the node state machine, the transport layer, the cost model, and the JSON and CSV configuration pipelines.

Chapter 4 presents the seven inventory policies, their mathematical formulations, and a comparative discussion of their strengths and limitations.

Chapter 5 describes the Optuna-TPE optimisation framework: search-space encoding, the fill-rate constraint mechanism, and the re-evaluation discipline.

Chapter 6 details the experimental design, dataset (M5 Walmart), warm-up exclusion, and multi-seed validation. It covers sixteen experiments: seven core optimisation experiments (E0–E4), four extension experiments (E5–E8), one stochastic robustness test (E9), and four preliminary validation experiments (EV1–EV4).

Chapter 7 presents results of all experiments with cost decompositions, fill-rate breakdowns, bullwhip ratios, and a cross-experiment synthesis table.

Chapter 8 summarises the contributions, quantifies the key findings, and outlines the future research roadmap.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the theoretical foundations underlying the study and situates the contribution within the existing literature. Seven areas are covered: single-echelon inventory theory, Clark-Scarf multi-echelon optimality, periodic-review and cyclic policies, the bullwhip effect, transport consolidation, simulation-optimisation methods, and existing open-source MEIO software.

2.1 SINGLE-ECHELON FOUNDATIONS

2.1.1 Newsvendor model

The **newsvendor model** (Arrow *et al.*, 1951) is the canonical single-period stocking problem. A retailer must commit to a stock level S before random demand D is realised; unsold units cost h per unit (holding) while unmet demand costs p per unit (shortage). The optimal stock level equates the critical ratio:

$$F(S^*) = \frac{p}{p + h} \quad (2.1)$$

where F is the CDF of demand. For Gaussian demand this yields the service-level formula $S^* = \mu + z_\alpha \sigma$, where z_α is the quantile of the standard normal corresponding to the target fill rate α .

2.1.2 Base-stock policy

The **base-stock (order-up-to) policy** extends the newsvendor to a multi-period setting with backlogging. Each period the stocking point orders up to a fixed target S , which is also the *inventory position* target. Under i.i.d. demand and deterministic lead time L (Zipkin, 2000), the optimal base-stock level is:

$$S^* = \mu(L + 1) + z_\alpha \sigma \sqrt{L + 1} \quad (2.2)$$

This formula is used to initialise the analytical baseline in experiment E0 (Section 7.2).

2.1.3 (s, S) and (R, Q) policies

The (s, S) **policy** places an order to restore inventory to level S whenever the inventory position falls below s (Scarf, 1960). It is optimal for single-echelon problems with fixed ordering costs under mild conditions. The (R, Q) **policy** places a fixed quantity Q whenever the inventory position reaches the reorder point R , which aligns naturally with discrete vehicle load sizes.

2.1.4 Economic Order Quantity (EOQ)

The **EOQ model** (Harris, 1913) minimises the sum of ordering and holding costs in a deterministic, continuous-review setting:

$$Q^* = \sqrt{\frac{2Kd}{h}} \quad (2.3)$$

where K is the fixed cost per order, d is the demand rate, and h is the holding cost rate. EOQ is a useful reference for calibrating order quantities in the (R, Q) policy but ignores service levels and capacity constraints.

2.2 MULTI-ECHELON OPTIMALITY — CLARK AND SCARF (1960)

Clark and Scarf (1960) established the foundation of MEIO by proving that, for a serial network with backlogging, independent demands, and infinite supplier capacity, the **echelon base-stock policy** is optimal.

The key insight is the *echelon inventory position* (EIP): rather than conditioning the replenishment decision at echelon i on only the local on-hand and in-transit stock, the

policy conditions on the stock at echelon i *plus* all downstream echelons. This accounts for the fact that inventory at echelon i will eventually flow downstream and must satisfy downstream demand as well as local demand.

The Clark-Scarf decomposition reduces the multi-echelon problem to a sequence of single-echelon newsvendor problems, one per echelon, solved in downstream-to-upstream order. The solution is optimal under the following assumptions:

- Serial (chain) topology — no divergent DCs serving multiple retailers.
- Stationary, independent demands at each echelon.
- No capacity constraints on orders or vehicles.
- Backlogging (no lost sales).

When these assumptions are violated — as they are in the divergent 1N3 topology with vehicle capacity and real M5 demand — the Clark-Scarf solution is no longer guaranteed to be optimal. This gap motivates the simulation-optimisation approach taken in this study.

2.3 PERIODIC-REVIEW AND CYCLIC POLICIES

2.3.1 (R, S) periodic-review policy

The (R, S) **policy** reviews inventory every R periods and orders up to level S . It reduces the administrative overhead of continuous-review policies and is widely used in practice. Optimal values of R and S can be found by exhaustive search or approximation (Silver, 1978).

2.3.2 (k, m) -cycle policies

Roundy (1985) showed that (k, m) -**cycle policies** — which allow at most k orders within an m -period cycle — can achieve near-optimal cost in multi-item inventory systems while enabling *co-ordinated replenishment*. By fixing the days on which each item is

reviewed, transport planners can schedule full-truckload deliveries in advance, reducing the empty-vehicle problem that plagues non-coordinated ordering.

2.4 THE BULLWHIP EFFECT

Lee *et al.* (1997) identified and formalised the **bullwhip effect**: the tendency for demand variability to amplify as orders propagate upstream in a supply chain. They attributed bullwhip to four root causes: demand signal processing (over-reaction to demand fluctuations), order batching, price fluctuations, and rationing games.

The most widely used empirical metric for bullwhip is the ratio of order variability to demand variability (Chen *et al.*, 2000), applied *per echelon*:

$$BW_{n,s} = \frac{CV^2(\text{orders placed by node } n \text{ for SKU } s)}{CV^2(\text{demand faced by node } n \text{ for SKU } s)} \quad (2.4)$$

A ratio greater than one at an upstream node indicates amplification. A subtlety, important for multi-echelon networks, is the choice of denominator: the “demand faced by node n ” is the *external* customer demand only when n is a retailer. At an internal node such as a warehouse, the relevant denominator is the time series of *aggregated child orders* arriving at n , not the raw external demand at any one downstream retailer (Lee *et al.*, 1997). Failing to aggregate produces an inflated, mis-attributed ratio at upstream nodes. This metric is computed per (node, SKU) on the post-warm-up window of each simulation run (Section 6.5).

2.5 TRANSPORT CONSOLIDATION IN INVENTORY MODELS

Classical MEIO models treat transport cost as a linear function of quantity shipped, ignoring the step-function cost structure of vehicle-based transport. In practice, a truck that is 50% full and a truck that is 100% full cost nearly the same to operate; the cost-per-unit is therefore roughly twice as high for the half-full truck.

Research on coordinated inventory and transport (see, e.g., Jagerman *et al.* 1990 and related work) showed that the interaction between inventory and transport decisions can be significant: coordinating order quantities with vehicle load sizes allows suppliers to dispatch fuller trucks, substantially reducing per-unit transport cost in realistic settings.

Despite this, the dominant MEIO literature — including Clark-Scarf, Graves-Willems, and most textbook treatments — ignores vehicle capacity and consolidation. This creates the gap that the joint optimisation framework in this study addresses: when transport cost dominates the budget, *inventory and transport decisions must be optimised jointly*.

2.6 SIMULATION-OPTIMISATION METHODS

When the performance surface of a stochastic or complex system cannot be expressed analytically, **simulation-optimisation** uses a simulator as an oracle and an optimisation algorithm to search the parameter space (Fu *et al.*, 2005). Various optimisation approaches are discussed here.

2.6.1 Classical gradient-free approaches

Genetic algorithms (GAs) (Goldberg, 1989) have been applied to MEIO parameter tuning since the 1990s. They are flexible and parallelisable but typically require thousands of function evaluations to converge and are not sample-efficient for smooth, low-dimensional surfaces. **Bayesian optimisation** (Shahriari *et al.*, 2016) addresses the sample-efficiency gap by fitting a Gaussian-process surrogate and using an acquisition function (e.g. expected improvement) to select the next evaluation point. It is highly sample-efficient for smooth functions in up to ~ 20 dimensions but scales cubically in the number of observations, making it slow for the 30-dimensional joint space studied here.

2.6.2 Tree-structured Parzen Estimator (TPE)

The **TPE algorithm** (Bergstra *et al.*, 2011) replaces the Gaussian-process surrogate with two separate kernel density estimators: one for the distribution of parameters that led to good results ($\ell(x)$) and one for the rest ($g(x)$). The acquisition function is $\ell(x)/g(x)$,

maximised by drawing many samples from $\ell(x)$ and selecting the one with the highest ratio.

TPE is well-suited to this study’s optimisation problem because:

- It handles mixed-integer variables (integer base-stock levels and continuous dispatch thresholds) natively.
- It scales approximately linearly in dimensionality, remaining tractable for the 30-dimensional joint search space (25 stock levels + 5 dispatch thresholds).
- It is implemented in OPTUNA (Akiba *et al.*, 2019), an open-source framework with support for parallelism and constraint handling.

2.6.3 Multi-objective optimisation: NSGA-II

When the goal is not to satisfy a hard service-level constraint but to *trace the trade-off* between cost and service level, a multi-objective evolutionary algorithm becomes appropriate. **NSGA-II** (Deb *et al.*, 2002) maintains a population of candidate solutions and uses non-dominated sorting and crowding-distance diversification to converge on the Pareto front in a single run. In this study, NSGA-II — implemented via Optuna’s NSGAIISampler — minimises $(C_{\text{total}}, -\hat{\alpha})$ jointly (i.e. minimises total cost while maximising fill rate $\hat{\alpha}$) and returns the non-dominated set as the Pareto front. This produces a smooth, monotone cost–service curve in a single run, replacing the earlier constraint-sweep approach in which the joint optimiser was re-run independently at each of several fill-rate targets; that constraint-sweep is retained as a sanity-check overlay (see Section 5.5).

2.6.4 Neural and reinforcement-learning approaches

Recent years have seen growing interest in applying **deep reinforcement learning** (DRL) to inventory control. Hubbs *et al.* (2020) introduce OR-GYM, an OpenAI-Gym-compatible benchmark suite that includes single-echelon and serial multi-echelon

environments; they show that policy-gradient methods (PPO, A2C) can match or exceed newsvendor-optimal policies in these settings. Gijsbrechts *et al.* (2022) demonstrate that DRL agents can learn near-optimal policies for the lost-sales problem with positive lead times — a notoriously intractable case for exact methods. Perez *et al.* (2021) extend DRL to stochastic serial systems, and Boute *et al.* (2022) provide a practitioner-oriented survey of DRL for supply-chain management.

Despite this progress, DRL approaches present several challenges for the problem studied here. First, the action space of the current framework is *mixed-integer*: base-stock levels are discrete (units) while dispatch thresholds are continuous. Policy-gradient methods require differentiable action distributions, which are poorly suited to large integer spaces without continuous relaxation. Second, DRL requires extensive environment interaction to learn a policy from scratch; the 30-dimensional joint search space in this study would require substantially more than 10^3 simulator evaluations to converge, rendering training time prohibitive within the scope of this study. Third, the parameters optimised by this framework are physically interpretable: base-stock levels map directly to safety-stock decisions and dispatch thresholds to consolidation strategy, supporting managerial interpretation of the result. TPE was therefore chosen over DRL for its sample-efficiency, mixed-integer native support, and the interpretability of its output.

2.7 EXISTING MEIO SOFTWARE

Several simulation and optimisation tools are available for supply-chain modelling, but none satisfies all requirements of this study simultaneously.

SIMPy A Python discrete-event simulation library. General-purpose and well-maintained, but provides no MEIO domain primitives — no nodes, policies, or transport layer. Building an MEIO simulator from SimPy requires implementing all domain logic from scratch.

ANYLOGIC A commercial simulation platform with a supply-chain library. Powerful and well-documented, but proprietary, expensive, and not scriptable for large-scale parameter sweeps.

SIMOPT An open-source Python library for simulation-optimisation (Pasupathy and Poloczek, 2022). Provides a collection of problem instances and solvers, but does not include MEIO-specific features such as multi-SKU demand, vehicle capacity, or transport consolidation.

OPTUNA An open-source Bayesian/TPE optimisation framework (Akiba *et al.*, 2019). Used as the outer optimisation loop in this study; does not include simulation functionality.

The tools surveyed above cover different parts of the problem well, but none combines an MEIO-specific simulation engine, a joint inventory-and-transport optimiser, and support for arbitrary DAG topologies and vehicle-capacity constraints in a single, open, scriptable package. This work builds such a framework and uses it to study the empirical value of joint optimisation on realistic demand data.

CHAPTER 3

DIGITAL-TWIN ARCHITECTURE

This chapter describes the design and implementation of the discrete-event simulation framework that forms the computational core of the work. Each section maps to one or more modules in the engine/ package and explains the algorithmic choices made at that layer.

3.1 NETWORK TOPOLOGY

The framework represents any supply-chain network as a **directed acyclic graph (DAG)**, implemented in `engine/network.py`. Each *node* in the graph corresponds to a stocking location (supplier, warehouse, or retailer) and each directed *edge* to a transport lane between two locations. Goods flow strictly downstream; returns and lateral transshipment are not modelled.

The topology is entirely configurable: the number of echelons (chain depth) and the number of nodes at each echelon (chain breadth) are controlled by the JSON configuration file without any change to simulator code. A single supplier feeding one warehouse and three retailers, a two-DC hub-and-spoke network, or a flat one-level distribution system are all expressible as DAG configurations. Throughout this work, several topologies of varying size and depth were used; the specific configuration for each experiment is described in Chapter 6.

Two further design choices are worth noting. First, the supplier node carries an optional `infinite_supply` flag: when set, the supplier can always fulfil any order placed by downstream nodes, representing an ideally reliable upstream source. This assumption is convenient for isolating the behaviour of downstream inventory policies but is

entirely optional — finite-capacity suppliers and supply disruptions are also supported (Section 3.7). Second, the DAG structure enforces a strict material-flow direction, ensuring that cycle detection is trivial and the topological processing order (Section 3.2) is unique.

3.2 DISCRETE-EVENT SIMULATION LOOP

The simulator advances time in unit steps of one day. Each day t executes the following sequence of operations:

1. **Receive shipments.** For every node, any shipment whose scheduled arrival time equals t is removed from the pipeline and added to on-hand inventory.
2. **Realise demand.** Each retailer node draws its external demand for day t from the configured demand generator (deterministic, Poisson, or CSV-driven).
3. **Clear external backlog.** Any accumulated external demand that could not be served on previous days is fulfilled from current on-hand stock, up to availability.
4. **Process child orders.** Internal orders from downstream nodes are served from the parent's on-hand inventory; unmet orders enter the parent's *children backlog*.
5. **Place replenishment orders.** Each non-supplier node queries its inventory policy to determine an order quantity and registers an IncomingOrder at its parent.
6. **Dispatch planning.** For each edge, the transport module inspects accumulated orders, builds vehicle loads respecting capacity constraints, and schedules shipments with the configured lead time (Section 3.4).
7. **Cost accumulation.** Holding, shortage, ordering, and transport costs for day t are recorded.

Figure 3.1 shows the corresponding control-flow diagram for a single daily tick.

Nodes are processed in *topological order* — retailers before warehouses, warehouses

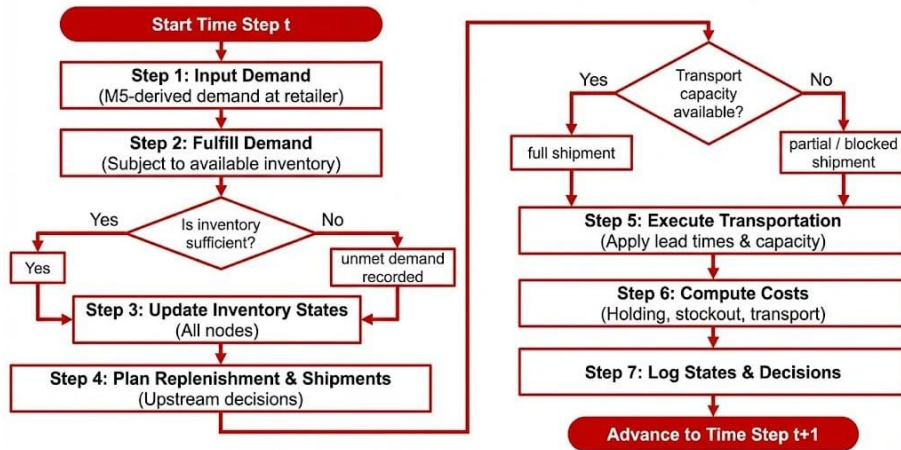


Figure 3.1: Discrete-event simulation loop for one daily tick.

before the supplier — so that each node’s child-order step is complete before its parent’s dispatch step. The topological sort is computed once at simulation startup (Simulator._topological_order).

The entry point is Simulator.run(mode), implemented in engine/simulator.py. When mode=”logs” the simulator records full daily time-series; when mode=”summary” only the aggregate KPI row is returned, which is sufficient for the optimisation inner loop.

Algorithm 1 summarises one daily tick.

3.3 NODE STATE MODEL

Each node in the network maintains the following per-SKU state variables, implemented as dictionaries keyed by SKU identifier in engine/node.py:

on_hand Current physical stock available for dispatch or external fulfilment.

pipeline_in A list of Shipment objects, each carrying a quantity and a scheduled arrival day. The total pipeline quantity is the sum of all in-transit units.

backlog_external Unfulfilled external demand accumulated over previous days; cleared before new demand is processed on each tick.

Algorithm 1 Daily simulation tick at time t

```
1: for each node  $n$  in topological order do
2:    $n.receive\_shipments(t)$ 
3:   if  $n$  is a retailer then
4:     demand  $\leftarrow$  demandGenerator[ $n$ ].get( $t$ )
5:      $n.process\_external\_demand(demand, t)$ 
6:   end if
7:    $n.clear\_external\_backlog()$   $\triangleright$  fill unmet external demand from prior days before
   internal orders
8:    $n.process\_child\_orders(t)$ 
9: end for
10: for each non-supplier node  $n$  in topological order do
11:    $q \leftarrow n.policy.order\_qty(on\_hand, backlog, pipeline\_in, t)$   $\triangleright q$ : order quantity in
   units
12:    $n.parent.add\_inbound\_order(n, q, t)$ 
13: end for
14: for each edge  $e$  in the network do
15:   loads  $\leftarrow$  GreedyLoadPlanner.plan( $e.pending\_orders$ )
16:   for each load  $\ell$  in loads do
17:     schedule shipment arrival at  $t + e.lead\_time$ 
18:     accumulateCost( $e, \ell$ )
19:   end for
20: end for
21: accumulateCost( $t$ )  $\triangleright$  holding and shortage
```

backlog_children A dictionary keyed by (child_id, sku) recording how many units the parent owes to each downstream child.

inbound_orders_queue Orders submitted by children that have not yet been dispatched.

placed_orders Historical record of orders placed by this node; used to compute the bullwhip ratio.

The two backlog dictionaries encode the dual role of intermediate nodes: they both *sell* to their children (backlog_children) and *buy* from their parent (backlog_external records their own unmet need).

3.3.1 Helper methods

Node.total_pipeline_in(sku) returns the scalar sum over all in-transit shipments for a given SKU, which is the quantity passed to the inventory policy as pipeline_in.

`Node.total_backlog_children(sku)` aggregates children-owed quantities across all child nodes, used by the echelon-stock calculation (Section 4.5).

3.4 TRANSPORT LAYER

Transport between nodes is modelled at the *vehicle load* level rather than at the individual unit level. The transport layer is implemented in `engine/transport.py` and consists of three collaborating types.

A key design feature is that a single lane (edge) between two nodes can carry **multiple transport options** simultaneously — one per available mode. For example, a warehouse-to-retailer lane might offer both a truck option (high capacity, 1-day lead time) and a rail option (higher capacity, 3-day lead time, lower cost per vehicle). This directly mirrors real-world logistics, where two locations are often served by more than one carrier type, each with its own cost structure and transit time. Multiple options are declared by listing several edge entries with the same from/to pair in the config's edges array, each with a distinct mode integer.

3.4.1 TransportOption

A `TransportOption` specifies the physical and economic properties of one transport mode on a lane:

- **mode** — a string label (e.g. "truck", "rail").
- **capacity** — maximum volume (in cubic metres) per vehicle.
- **weight_capacity** — maximum gross weight (kg) per vehicle; `None` means no weight constraint.
- **cost_full / cost_half / cost_quarter** — three billing tiers representing real-world freight service categories. The `GreedyLoadPlanner` applies them as follows:
 - (i) every vehicle packed to full capacity in the greedy loop is charged `cost_full` (dedicated full-truckload, FTL);
 - (ii) a partial remaining vehicle with utilisation

$\geq 85\%$ is also charged `cost_full` — at near-full load, booking a dedicated truck is the standard industry practice (the typical FTL/LTL crossover lies at 85–90%); (iii) a partial vehicle with utilisation in the range $[50\%, 85\%)$ is charged `cost_half` (consolidated or less-than-truckload freight); (iv) utilisation below 50% is charged `cost_quarter` (parcel or courier service). Within each tier the fee is flat, so there is no marginal cost signal pushing the optimiser to delay dispatch to fill the remaining few percent — dispatch timing is governed solely by the `min_dispatch_utilization` threshold (see below).

- **lead_time** — deterministic transit days (stochastic lead times are a planned extension).
- **min_dispatch_utilization** — minimum fraction of capacity that must be filled before a vehicle is released. A value of 0 means dispatch on any positive load; a value of 0.8 means the planner waits until 80% of capacity is committed.

When multiple `TransportOptions` exist on a lane, the framework’s default **mode selector** sorts them by their mode integer (1 = primary, 2 = secondary, . . .) and dispatches all cargo via the lowest-numbered option, with higher-numbered modes serving as fallbacks. In addition, the optional `share` field on an edge enables *weighted random sampling* of lead times across options — for example, assigning `share: 0.8` to a truck route and `share: 0.2` to a rail route means 80% of dispatch events use the truck lead time and 20% use the rail lead time, directly modelling realistic carrier variability for multi-route logistics corridors without splitting individual vehicle loads.

3.4.2 GreedyLoadPlanner

The `GreedyLoadPlanner` is the load-packing algorithm used in all experiments in this work. It is the default implementation of the `LoadPlanner` protocol. Given a list of pending order lines for a lane, it greedily packs items into vehicles one at a time, subject to dual constraints:

1. **Volume constraint:** total volume of packed items $\leq$ capacity.
2. **Weight constraint:** total weight of packed items $\leq$ weight_capacity (when set).

If weight capacity is tighter than volume capacity for the current mix of SKUs, the effective capacity is scaled down accordingly before packing begins. After packing, if the resulting utilisation is below `min_dispatch_utilization`, the load is *held* for up to `max_dispatch_wait` days; once the wait limit is reached the load is dispatched regardless of fill level to prevent indefinite stockouts at downstream nodes.

The tiered cost is evaluated after packing: loads below 50% utilisation pay `cost_quarter`; loads in [50%, 85%) pay `cost_half` (consolidated LTL freight); loads at or above 85% — including exactly full loads — pay `cost_full` (dedicated FTL rate, consistent with the industry crossover described above). This incentivises consolidation but never blocks dispatch beyond the wait limit. Utilisation is computed relative to the effective capacity — the lesser of the raw volume capacity and the weight-implied volume limit — so the 50% and 85% tier boundaries apply consistently regardless of which constraint binds.

3.4.3 Pluggable protocols

The transport layer is designed around three narrow protocols, making all three components fully **plug-and-play**:

LoadPlanner Decides how pending orders are packed into vehicle loads.

GreedyLoadPlanner is the default used in this work; alternatives (bin-packing, probabilistic packing) can be substituted by implementing the same two-method interface.

ModeSelector Chooses which transport option(s) to activate when multiple options exist on a lane. The default CheapestModeSelector always picks the primary mode (lowest mode integer); a custom selector could split loads across modes or apply routing rules.

CostAllocator Distributes a vehicle's total cost across the SKUs it carries. The default allocates cost proportionally by volume.

No changes to the simulator core (engine/simulator.py) are required to swap any of these components — they are injected at construction time, making the transport layer independently extensible.

3.5 COST MODEL

The total cost accumulated over the simulation horizon T is:

$$C_{\text{total}} = C_{\text{hold}} + C_{\text{transport}} + C_{\text{order}} + C_{\text{shortage}} \quad (3.1)$$

Each component is defined as follows.

Holding cost.

$$C_{\text{hold}} = \sum_{t=w}^T \sum_n \sum_s h_{n,s} \cdot \text{on_hand}_{n,s,t} \quad (3.2)$$

where $h_{n,s}$ is the per-unit-per-day holding cost for SKU s at node n and w is the warm-up cutoff.

Shortage (backlog) cost.

$$C_{\text{shortage}} = \sum_{t=w}^T \sum_n \sum_s p_{n,s} \cdot \text{backlog_external}_{n,s,t} \quad (3.3)$$

where $p_{n,s}$ is the per-unit-per-day penalty cost.

Ordering cost. Each order placed by a node incurs a fixed setup cost K_n plus a per-unit variable cost $c_{n,s}$:

$$C_{\text{order}} = \sum_{\text{orders}} (K_n + c_{n,s} \cdot q) \quad (3.4)$$

where q is the order quantity in units.

Transport cost. Each dispatched vehicle load on edge e with utilisation u incurs a tiered cost:

$$C_{\text{transport}}(e, u) = \begin{cases} \text{cost_full} & \text{if } u \geq 0.85 \\ \text{cost_half} & \text{if } 0.5 \leq u < 0.85 \\ \text{cost_quarter} & \text{if } u < 0.5 \end{cases} \quad (3.5)$$

All four components are accumulated on every tick inside `Simulator.run` and the first w days are excluded from reported KPIs to avoid contamination by the initial-inventory transient (Section 6.4).

3.6 CONFIGURATION AND INPUT METHODS

Scenarios are specified as JSON files. The top-level keys are:

seed Integer random seed for reproducibility.

time_horizon Number of simulation days.

skus List of SKU identifiers.

sku_properties Per-SKU `volume_per_unit` and `weight_per_unit`.

nodes List of node objects specifying type (supplier/warehouse/retailer), initial inventory, cost parameters, and per-SKU policy configuration.

edges List of edge objects specifying from/to nodes and a list of `TransportOption` specifications.

disruptions (Optional) List of supply disruption events; each event specifies a node, a start day, and a duration.

The engine/config_validator.py module validates all required fields and raises descriptive errors before the simulation starts, preventing silent misconfiguration during long experiment runs.

In addition, every experiment run emits a topology sanity line at startup of the form Topology : 1 supplier, 2 warehouses, 3 retailers [Supplier $\rightarrow$ W1,W2 $\rightarrow$ R1,R2,R3] so that misreadings of the network shape (e.g. “1N3” interpreted as a 1-1-3 chain rather than a 1-2-3 with $N = 2$ warehouses) are caught at the start of every run.

Randomness in the framework arises from two sources: (i) stochastic demand generators — for example, Poisson-distributed daily demand draws a fresh random variate each day — and (ii) the Optuna TPE optimiser, which uses probabilistic surrogate models and random sampling to explore the parameter space. The seed value initialises Python’s random module and NumPy’s RNG at simulation startup, ensuring that both demand sequences and optimiser trial sequences are exactly reproducible across runs. CSV-driven demand (used in all main experiments of this work) is fully deterministic and unaffected by the seed; the seed matters primarily when using Poisson or other synthetic demand modes and when running the optimiser.

3.6.1 CSV input pipeline

To lower the barrier for practitioners who are not comfortable editing JSON directly, a secondary input method was developed using six CSV files. The builder script inputs/scripts/build_config_from_csv.py reads and validates all six files and emits a complete JSON configuration that can then be passed to any simulation script.

Table 3.1 lists the six files and their purposes.

Per-retailer daily demand is read from CSV files in inputs/demand_data/, derived from the M5 Walmart Forecasting dataset (Makridakis *et al.*, 2022).

Table 3.1: CSV input files and their roles in the configuration pipeline.

File	Contents
sim.config.csv	Global parameters: seed, horizon, warm-up, strategy
products.csv	SKU physical properties: volume and unit cost
nodes.csv	One row per node — type, inventory, cost parameters
policies.csv	Long-format policy parameters (one row per parameter)
routes.csv	Transport lanes: capacity, tiered costs, lead time
demand_config.csv	Maps each retailer to its demand CSV

3.7 SUPPLY DISRUPTION MODELLING

Supply disruptions are modelled by temporarily blocking a node from receiving or dispatching stock for a specified interval $[t_s, t_s + d)$. During a disruption, orders placed by downstream nodes accumulate in the `backlog_children` dictionary of the affected node. When the node recovers at day $t_s + d$, it resumes normal order fulfilment and works through the accumulated backlog.

Disruptions are declared in the `disruptions` list of the config JSON. Multiple, non-overlapping disruption events can be specified per node. The disruption model is deliberately simple — it does not include rerouting or mode switching — making it straightforward to extend.

CHAPTER 4

INVENTORY POLICIES

Seven inventory control policies are implemented in the `policies/` package, covering the major families found in the MEIO literature. They share a common interface — every policy exposes a single `order_qty` method:

Listing 4.1: Universal policy interface.

```
def order_qty(  
    self,  
    on_hand: int,  
    backlog_external: int,  
    backlog_children: int,  
    pipeline_in: int,  
    t: Optional[int] = None,  
) -> int:  
    ...
```

The `t` argument is the simulation day; periodic policies use it to determine whether a review is due, while continuous-review policies ignore it. The *inventory position* (IP) used in most policies is:

$$\text{IP} = \text{on_hand} - \text{backlog_external} + \text{pipeline_in} \quad (4.1)$$

Orders are always non-negative; over-stocking is never triggered.

4.1 BASE-STOCK POLICY

Implementation: `policies/base_stock.py`

The base-stock (order-up-to) policy is the single-parameter benchmark derived from

newsvendor theory. On every period, the order quantity is:

$$q = \max(0, S - \text{IP}) \quad (4.2)$$

where S is the *base-stock level*. The optimal S under Gaussian demand with mean μ and standard deviation σ over a lead time L is the newsvendor quantile:

$$S^* = \mu(L + 1) + z_\alpha \sigma \sqrt{L + 1} \quad (4.3)$$

where z_α is the standard-normal quantile corresponding to the target fill rate α (e.g. $z_{0.92} \approx 1.41$). This formula is used to initialise the analytical baseline in experiment E0.

The base-stock policy is *optimal for single-echelon, uncapacitated, backlog models* but does not account for transport capacity, vehicle consolidation, or order batching.

4.2 REORDER-POINT FAMILY: (s, S) AND (R, Q)

Implementations: policies/ss_policy.py, policies/rq_policy.py

4.2.1 (s, S) policy

The (s, S) policy is a continuous-review policy with two parameters: a reorder point s and an order-up-to level $S > s$. An order is placed only when the inventory position falls at or below s :

$$q = \begin{cases} S - \text{IP} & \text{if } \text{IP} \leq s \\ 0 & \text{otherwise} \end{cases} \quad (4.4)$$

Unlike the base-stock policy, (s, S) batches demand between review events, which naturally aligns order sizes with vehicle loads. This batching behaviour was one motivation for including it alongside base-stock in the preliminary validation experiments

(Section 6.1.3).

4.2.2 (R, Q) policy

The (R, Q) (reorder-point, fixed-quantity) policy places a fixed order of size Q whenever the inventory position drops to or below the reorder point R :

$$q = \begin{cases} Q & \text{if IP} \leq R \\ 0 & \text{otherwise} \end{cases} \quad (4.5)$$

Because Q is fixed, orders are always the same size — well-suited to lanes with a fixed vehicle capacity.

4.3 PERIODIC-REVIEW: (R, S) AND ORDER-UP-TO WITH PHASE OFFSET

Implementations: policies/order_up_to.py, policies/periodic_review.py

4.3.1 (R, S) periodic-review policy

The PeriodicReviewPolicy checks inventory every R days and orders up to level S :

$$q = \begin{cases} \max(0, S - \text{IP}) & \text{if } t \bmod R = 0 \\ 0 & \text{otherwise} \end{cases} \quad (4.6)$$

Between reviews, no order is placed regardless of the inventory position. The periodic structure reduces ordering overhead and can be co-ordinated across nodes to smooth vehicle utilisation.

4.3.2 Order-up-to with phase offset

The OrderUpToPolicy generalises the above by allowing a `phase_offset`: reviews occur when $(t - \text{phase_offset}) \bmod R = 0$. This lets different nodes review on staggered days, spreading transport demand across the week and improving consolidation opportunities.

Optional k and m parameters further restrict the maximum number of orders (k) within a

sliding window of m days, combining periodic review with a (k, m) -cycle cap.

4.4 CYCLIC POLICY: (k, m) -CYCLE SCHEDULING

Implementation: policies/km_cycle.py

The KmCyclePolicy schedules at most k replenishment orders per m -day cycle, at pre-specified intra-cycle offsets:

$$q = \begin{cases} \max(0, S - \text{IP}) & \text{if } t \bmod m \in \text{review_offsets} \text{ and orders this cycle} < k \\ 0 & \text{otherwise} \end{cases} \quad (4.7)$$

For example, with $m = 7$ and $\text{review_offsets} = \{0, 3\}$, the policy reviews on Monday and Thursday of each week. This structure is designed to be *consolidation-friendly*: if the warehouse always orders on fixed days, the supplier can prepare full truck loads in advance.

4.5 ECHELON BASE-STOCK (CLARK-SCARF)

Implementation: policies/echelon_stock.py

The echelon base-stock policy implements the optimal policy for serial multi-echelon systems identified by Clark and Scarf (1960). Rather than conditioning on the local inventory position, it conditions on the *echelon inventory position* (EIP) — the stock at the current node plus all downstream nodes, both on-hand and in-pipeline:

$$\text{EIP}(n) = \text{on_hand}(n) + \sum_{d \in \text{downstream}(n)} [\text{on_hand}(d) + \text{pipeline_in}(d)] \quad (4.8)$$

The order quantity is:

$$q = \max(0, S_e - \text{EIP}(n)) \quad (4.9)$$

where S_e is the echelon base-stock level. The simulator computes the EIP via a DFS traversal of the downstream subgraph in `Simulator._echelon_pipeline`, and passes the result to the policy through the `pipeline_in` argument.

The echelon stock policy is optimal under the restrictive assumptions of Clark and Scarf (serial network, independent demands, no capacity constraints), which are *not* fully satisfied in the 1N3 topology. Nevertheless, it provides an important upper bound on what a classical analytical approach can achieve.

4.6 POLICY COMPARISON

Table 4.1 summarises the seven policies along four dimensions: number of parameters, review type, whether the policy accounts for network structure, and its amenability to mixed-integer optimisation.

Table 4.1: Summary of the seven implemented inventory policies.

Policy	Params	Review	Network-aware	Opt-friendly
Base-stock	1	Continuous	No	Yes
(s, S)	2	Continuous	No	Yes
(R, Q)	2	Continuous	No	Yes
(R, S) periodic	2	Periodic	No	Yes
Order-up-to + offset	2–4	Periodic	No	Yes
(k, m) -cycle	3–4	Periodic	No	Yes
Echelon base-stock	1	Continuous	Yes	Yes

All policies are parameterised by small integer or continuous scalars, making them amenable to the Optuna-TPE optimiser described in Chapter 5. The echelon base-stock policy is the only policy that explicitly accounts for downstream inventory, at the cost of requiring the simulator to compute the EIP on every tick.

The main optimisation experiments (E0–E4) use the base-stock policy for all nodes because its single-parameter structure keeps the search space tractable (25 parameters for $5 \text{ SKUs} \times 5 \text{ non-supplier nodes}$ — W1, W2, R1, R2, R3) while being flexible enough to demonstrate the joint optimisation benefit. The policy comparison experiment (E6, Section 7.10) re-runs the optimisation with (s, S) , periodic-review, and echelon base-stock as alternatives.

CHAPTER 5

OPTIMISATION FRAMEWORK

This chapter describes the Optuna-TPE-based optimisation framework (optimizer/optimize.py) used to search for high-quality inventory and transport parameters. Three modes are supported: inventory-only, transport-only, and joint. The joint mode is the central research contribution of this study.

5.1 NEED FOR OPTUNA TPE

The optimisation problem of selecting base-stock levels and dispatch thresholds for a multi-echelon network is a *black-box, mixed-integer, constrained* problem: the objective function (total simulated cost subject to a fill-rate constraint) has no closed-form gradient, the decision variables include both integer-valued stock levels and continuous-valued dispatch thresholds, and the feasible region is defined by a fill-rate inequality that cannot be expressed analytically.

Classical derivative-based methods are inapplicable. Genetic algorithms and simulated annealing are gradient-free but require many evaluations to converge. Bayesian optimisation with a Gaussian-process surrogate is sample-efficient but scales poorly beyond ~ 20 dimensions due to the cubic cost of kernel inversion.

The **Tree-structured Parzen Estimator (TPE)** algorithm addresses these limitations:

- It models the distribution of good and bad parameters separately using kernel density estimators, requiring no global surrogate.
- It scales approximately linearly in the number of dimensions, making it practical for 25–30 dimensional joint spaces.

- It handles mixed-integer variables natively — integer and continuous parameters are treated uniformly.
- It is implemented in the open-source OPTUNA library with support for parallelism and pruning.

5.2 SEARCH-SPACE ENCODING

5.2.1 Inventory: base-stock levels

For the inventory mode, the optimiser searches over the base-stock level $S_{n,s}$ for every non-supplier (node, SKU) pair. In the 1N3 topology with $N = 2$ warehouses (W1, W2) and 3 retailers (R1, R2, R3) — five non-supplier nodes in total — and 5 SKUs, this gives $5 \times 5 = 25$ integer parameters.

Each parameter $S_{n,s}$ is bounded relative to the analytical base-stock level $S_{n,s}^*$ (Equation 4.3):

$$S_{n,s} \in \left[\max(10, \lfloor 0.3 \cdot S_{n,s}^* \rfloor), \lceil 3.0 \cdot S_{n,s}^* \rceil \right] \cap \mathbb{Z} \quad (5.1)$$

The lower bound of 10 units prevents degenerate stock-out configurations. The upper bound of $3 \times S^*$ is wide enough to encompass very conservative safety-stock strategies.

5.2.2 Transport: dispatch thresholds

For the transport mode, the optimiser searches over the min_dispatch_utilization threshold δ_e for every edge e in the network. The 1N3 topology has 5 directed edges (Supplier→W1, Supplier→W2, W1→R1, W1→R2, W2→R3), giving 5 continuous parameters:

$$\delta_e \in [0.0, 0.9] \quad (5.2)$$

A value of 0 means dispatch on any positive load; a value of 0.9 means the planner waits

until 90% of the vehicle’s capacity is committed. The upper bound of 0.9 (rather than 1.0) prevents the edge from deadlocking when demand is low.

5.3 JOINT SEARCH

In joint mode, a single Optuna study is created over the *union* of the inventory and transport search spaces — 30 parameters for the 1N3 ($N = 2$) topology (25 integer stock levels + 5 continuous dispatch thresholds). A single trial proposes all 30 values simultaneously, evaluates the full simulation, and returns the penalised cost.

This differs from an alternating or decomposed optimisation approach: both sets of parameters are updated together in each trial, allowing the TPE model to learn cross-parameter interactions — specifically, that raising $S_{n,s}$ (inventory) enables raising δ_e (consolidation) without increasing stockout risk.

The joint study runs n_{trials} sequential trial evaluations by default.

5.4 CONSTRAINED OPTIMISATION — MINIMUM FILL RATE

Most supply chains operate under a minimum service-level agreement. The optimiser enforces a minimum fill rate $\alpha_{\min}$ via a *soft penalty*:

$$\text{objective} = C_{\text{total}} + \max(0, \alpha_{\min} - \hat{\alpha}) \times 100 \times 10^6 \quad (5.3)$$

where $\hat{\alpha}$ is the fill rate achieved by the current trial and the 10^6 multiplier per percentage-point shortfall makes a 1% fill-rate violation cost \$1 million — far exceeding any realistic cost saving. This ensures that feasible trials (those with $\hat{\alpha} \geq \alpha_{\min}$) always dominate infeasible ones in the Pareto ranking.

After all trials are complete, the best *feasible* trial is selected as the result. If no trial satisfies the fill-rate constraint, the framework falls back to the best-overall trial and

warns the user.

5.4.1 Aggregate vs per-SKU constraint

Equation 5.3 enforces the constraint on the *aggregate* fill rate, allowing high-volume SKUs to over-serve and compensate for under-served low-volume SKUs. In service-level agreements that guarantee fill rate *per item*, the constraint should bind for every SKU individually. The optimiser supports this via a `per_sku_constraint=True` flag, which replaces the scalar penalty with a per-SKU sum:

$$\text{penalty}_{\text{per-SKU}} = \sum_{s \in \text{SKUs}} \max(0, \alpha_{\min} - \hat{\alpha}_s) \times 100 \times 10^6 \quad (5.4)$$

A trial is feasible only when every SKU's fill rate meets the target. Experiment E3_ per_sku (Section 7.7) re-runs the joint optimisation under this stricter mode and reports the cost premium.

5.5 PARETO EXPLORATION

Experiment E4 maps the cost–service Pareto frontier by two complementary methods: NSGA-II (primary) and a constraint sweep (sanity check).

5.5.1 Primary: NSGA-II two-objective study

The Optuna NSGAIISampler is run on the same 30-dimensional joint search space, but with two minimisation objectives:

$$\min(C_{\text{total}}, -\hat{\alpha}) \quad (5.5)$$

After 200 trials the non-dominated trials in `study.best_trials` form the Pareto front. This produces a smooth, monotone cost–service trade-off curve in a single run, without needing to choose fill-rate target points up-front. The NSGA-II frontier is the primary

result reported in Chapter 7.

5.5.2 Sanity check: constraint sweep

The original constraint-sweep procedure is retained as a sanity-check overlay. The joint optimiser is re-run at eight integer-step targets $\alpha_{\min} \in [92\%, 99\%]$, each independent and seeded differently. At lower targets the constraint is non-binding (the optimiser naturally finds higher fill rates) so those points do not lie on the true Pareto front; the trimmed range $\geq 92\%$ ensures the constraint actually binds.

5.6 RE-EVALUATION AND SEARCH SPACE SUMMARY

Optuna internally minimises a noisy objective computed from a single simulation run per trial. Because the demand series is deterministic (seeded) in these experiments, the noise comes only from the trial-to-trial variation in dispatch decisions under different threshold values. However, for statistical rigour, the best parameter set identified by the optimiser is *re-evaluated* in a fresh simulation run before its cost and fill rate are reported.

The re-evaluated metrics — not the optimiser’s internal trial value — are what appear in Chapter 7. The best parameters are also saved to a `params.json` file in the experiment output directory, enabling reproducibility without re-running the optimiser.

5.6.1 Search space dimensionality

Table 5.1 summarises the search space for each optimisation mode on the 1N3 ($N = 2$), 5-SKU network.

The 30-dimensional joint space is well within the practical range of TPE-based Bayesian optimisation (commonly cited as up to ~ 50 dimensions). Empirically, per-trial wall-clock time is flat from 5 to 30 dimensions (127–140 ms/trial), confirming that TPE bookkeeping overhead is negligible relative to the 365-day simulation evaluation; 100 trials require approximately 13 seconds per experiment. Full per-experiment runtimes and peak memory measurements are reported in Appendix B.

Table 5.1: Search space dimensionality by optimisation mode (1N3 with $N = 2$ warehouses, 5 SKUs).

Mode	Integer params	Continuous params	Total
Inventory-only	25	0	25
Transport-only	0	5	5
Joint	25	5	30
E6 base_stock (joint)	25	5	30
E6 (s, S) (joint)	$25+5\alpha$	5	35
E6 periodic_review (joint)	$25+5R$	5	35
E6 echelon_stock (joint)	25	5	30

Per-policy parameter registry (E6). Experiment E6 (Section 7.10) compares four policy families, each jointly optimising its inventory parameters *and* all five dispatch thresholds, so that every policy has equal access to transport consolidation benefits. The per-policy registry in `optimizer/policy_search.py` extends the base inventory space: (s, S) adds an $\alpha = s/S$ ratio per node (one shared across SKUs at each node); periodic-review adds a review period R per node; echelon base-stock reuses a single level. All variants then include the five continuous dispatch thresholds D_{lane} , giving effective search spaces of 30 dimensions (base_stock, echelon_stock) and 35 dimensions (ss, periodic_review).

Bullwhip-aware joint optimisation (E8). Experiment E8 (Section 7.12) augments the joint objective with a bullwhip term:

$$\min C_{\text{total}} + \lambda \cdot \overline{\text{BW}} + \text{penalty}(\hat{\alpha}) \quad (5.6)$$

where $\overline{\text{BW}}$ is the mean bullwhip ratio and λ trades off cost against variance amplification.

The three optimisation modes — inventory-only, transport-only, and joint — together with the NSGA-II Pareto study, the per-SKU constraint mechanism, and the bullwhip-aware objective, form a complete optimisation framework. Experimental results obtained using this framework are presented in Chapter 7.

CHAPTER 6

EXPERIMENTAL METHODOLOGY

This chapter describes the experimental design, datasets, and evaluation methodology used in this study. The experiments are organised into three groups: four preliminary validation experiments (EV1–EV4) on simplified single-SKU topologies, seven core optimisation experiments (E0–E4 plus E3_per_sku) on the full 5-SKU 1N3 network, and five extension and robustness experiments (E5–E8 plus E9) that stress-test the joint optimum under disruption, alternative inventory policies, demand-forecast error, bullwhip-aware objectives, and stochastic demand.

6.1 PRELIMINARY VALIDATION EXPERIMENTS (EV1–EV4)

Before running the main optimisation suite, four controlled experiments were designed to verify that the simulator correctly reproduces known theoretical behaviour. EV1–EV3 used a simplified **single-SKU, 1-1-1** topology (1 supplier $\rightarrow$ 1 warehouse $\rightarrow$ 1 retailer) with synthetic demand (constant 5 units/day for EV1 and EV3; a constant background with a single spike at day 20 for EV2). EV4 scaled to a single-SKU **1-2-3** topology (1 supplier $\rightarrow$ 2 warehouses $\rightarrow$ 3 retailers) and used real M5 Walmart demand data.

6.1.1 EV1 — Constant-demand convergence

A constant daily demand of d units is applied to all retailers over a 100-day horizon. Each node runs a base-stock policy with $S = d(L + 1)$ where L is the lead time from its parent. The test confirms that on-hand inventory at each node converges to and stays at $S - d \cdot L$ (the equilibrium on-hand level) within $L + 1$ days.

6.1.2 EV2 — Demand shock response

A synthetic demand series with constant background demand d and a single large spike at day 20 is read. The experiment verifies that the upstream nodes observe an amplified

Table 6.1: Preliminary validation experiments: scope and purpose.

ID	Title	Purpose
EV1	Constant-demand convergence	Verify that base-stock policy drives on-hand inventory to the target level under deterministic constant demand.
EV2	Demand shock response	Verify that a one-time spike propagates upstream with pipeline delay and amplification.
EV3	Policy comparison under transport capacity	Compare base-stock vs (s, S) with and without vehicle capacity limits; confirm batching effect.
EV4	Capacity scaling on M5 data	Quantify the non-linear cost–service trade-off as transport capacity is scaled from $0.25\times$ to $4\times$ baseline.

Table 6.2: Network topologies and demand configurations used across all experiments.

Experiment group	Topology	SKUs	Demand
EV1–EV3	1-1-1	1	Synthetic (constant or single spike)
EV4	1-2-3	1	M5 Walmart (historical)
E0–E8 (main)	1-2-3	5	M5 Walmart (historical)
E9 (stochastic)	1-2-3	5	Poisson ($\lambda = \text{M5 mean per retailer/SKU}$)

order spike with a delay matching the cumulative pipeline lead time.

6.1.3 EV3 — Policy comparison under transport capacity

Two policies (base-stock and (s, S)) are compared on the single-SKU topology with and without a finite vehicle capacity. With no capacity limit both policies track demand similarly. With capacity limits, the (s, S) fixed-quantity orders align better with vehicle loads.

6.1.4 EV4 — Capacity scaling

Vehicle capacity is scaled by multipliers $\{0.25, 0.5, 1.0, 2.0, 4.0\}$ relative to a baseline calibrated to carry one week of average demand. M5 Walmart demand data is used for realism. Results are shown in Table 7.1 (Section 7.1.4).

6.2 DATASET: M5 WALMART FORECASTING DATA

The demand data for all experiments is derived from the **M5 Walmart Forecasting dataset**. The M5 dataset contains daily unit sales for thousands of products across ten Walmart stores in the United States, spanning approximately five years.

The M5 dataset was selected because it exhibits realistic retail demand characteristics that stress inventory policies beyond what synthetic Poisson processes can capture:

- **Intermittency and spikes.** Sales often cluster around promotional events and holidays, creating irregular demand patterns.
- **Weak weekly seasonality.** Weekend peaks and weekday troughs introduce low-frequency variation.
- **Long tails.** Occasional large orders produce high coefficient-of-variation values that challenge base-stock stability.

Per-retailer daily demand CSVs were derived by selecting one M5 series per retailer per SKU. For the main experiments, the simulator reads the *last 365 days* of each series, which corresponds to the most recent year of Walmart operations in the M5 timeline. This choice avoids the non-stationary trend present across the full 1913-day series while keeping the 365-day horizon long enough for fill-rate statistics to stabilise. For the preliminary validation experiments, a separate synthetic series with a single spike at day 20 was constructed.

6.3 MAIN EXPERIMENT CONFIGURATION

All main experiments (E0–E9) share the following common parameters:

Table 6.3: Deterministic lead times in the canonical 1N3 configuration.

Lane	Lead time
Supplier → W1	5 days
Supplier → W2	6 days
W1 → R1	2 days
W1 → R2	3 days
W2 → R3	4 days

Table 6.4: Main experiment simulation parameters.

Parameter	Value
Network topology	1 supplier, 2 warehouses (W1, W2), 3 retailers (R1–R3)
Service map	W1 → {R1, R2}; W2 → R3
SKUs	5 (SKU_A through SKU_E)
Simulation horizon	365 days
Demand window	M5 days 1548–1912 (most recent 365 days), strategy=clip
Warm-up period	30 days (per-experiment override via recommended_warmup)
Demand type	CSV-driven (M5 Walmart, retailer-specific)
Fill-rate target	≥ 92% (soft constraint, experiments E1b onwards)

SKU physical properties span a range of volumes (0.002–0.008 m³/unit) and weights (0.5–1.2 kg/unit), so the GreedyLoadPlanner’s dual volume-and-weight capacity constraint is active for some SKU mixes.

6.4 EXPERIMENTAL CONTROLS

At simulation start, all nodes are initialised with a configurable amount of stock. This initial stock creates a transient period during which KPIs reflect the initialisation state

rather than the steady-state behaviour of the policy. To avoid this bias, an initial window of each simulation run is *excluded* from all reported KPIs (costs, fill rates, bullwhip ratios). Inventory dynamics still execute during the warm-up window so the system reaches steady state before measurement begins.

The warm-up length is chosen *per experiment* via the recommended `warmup` field on each entry of `EXPERIMENT_META` in the runner script. For the M5 365-day scenario, this is 30 days — approximately 8% of the horizon and at approximately three times the longest pipeline lead time (Supplier→W2 at 6 days plus W2→R3 at 4 days, totalling 10 days). The CLI `-warmup` flag overrides this per run; synthetic configurations whose transient itself is the signal (e.g. the demand-shock validation EV2) pass `-warmup 0` explicitly. The runner emits a warning if a non-trivial scenario ($T \geq 200$) is run with no warm-up.

6.4.1 Multi-seed statistical validation

Multi-seed validation is integrated into the experiment runner via the `-validate_seeds N` flag. When supplied, the runner re-evaluates the best E3 parameter set in N fresh simulations whose random seeds are advanced by one each time, and writes `multiseed_validation.csv` (one row per seed) and `multiseed_summary.csv` (mean and standard deviation of total cost and fill rate).

Because the main experiments use a fixed deterministic M5 demand CSV *and* deterministic lead times, demand realisations do not vary across seeds; the multi-seed run therefore reports zero variance under the canonical configuration. Cross-seed variance becomes informative when the demand generator is Poisson or the lead-time generator is stochastic, which the framework supports but the canonical 1N3 5-SKU scenario does not exercise.

6.5 BULLWHIP METRIC

The bullwhip ratio is computed *per echelon* as the ratio of the order coefficient-of-variation squared at node n to the coefficient-of-variation squared of the time series of orders *arriving at* node n :

$$\text{BW}_{n,s} = \frac{\text{CV}^2(\text{orders out of } n \text{ for SKU } s)}{\text{CV}^2(\text{orders into } n \text{ for SKU } s)} \quad (6.1)$$

The “orders into n ” series is defined as

- the external customer demand series, when n is a retailer;
- the daily sum of orders_to_parent from every child of n , when n is a warehouse or the supplier.

Aggregating child orders is essential for upstream nodes: a warehouse with two retailers as children faces the *sum* of their orders, not the demand at any one retailer. Failing to aggregate produces an inflated, mis-attributed ratio. This per-echelon definition follows Lee *et al.* (1997) and is verified by a hand-computable scenario in tests/test_bullwhip.py. Each row of the resulting bullwhip.csv stores node_id, sku, cv2_in, cv2_out, and bullwhip_ratio so the two component variances can be inspected independently. The metric is computed on the post-warm-up window only.

6.6 MAIN OPTIMISATION EXPERIMENTS E0–E9

Twelve experiments cover the full study; seven form the core (E0–E4 and E3_per_sku), four (E5–E8) extend the canonical optimum to disruption, alternative policies, forecast error, and bullwhip-aware objectives, and one (E9) tests stochastic demand robustness. Table 6.5 summarises each.

All experiments except E0 use the Optuna-TPE optimiser described in Chapter 5. E4

Table 6.5: Main optimisation experiments: design and hypothesis or finding.

ID	Title	Hypothesis or finding
E0	Analytical baseline (newsvendor)	Newsvendor stock + zero dispatch threshold leaves transport cost dominant.
E1a	Fixed 25% dispatch threshold	Naive consolidation collapses fill rate.
E1b	Transport-only optimisation	Cannot reach 92% with inventory fixed at E0 levels.
E2	Inventory-only optimisation	Reaches the fill target but cost rises.
E3	Joint optimisation	Lowest cost at $\geq 92\%$ fill.
E3_per_sku	Joint with <i>per-SKU</i> fill constraint	Removes worst-SKU shortfall at small cost premium.
E4	Pareto frontier (NSGA-II + sweep)	Diminishing returns above 95% fill.
E5	Disruption robustness (14-day W1 outage)	E3 parameters provide greater resilience; the optimiser is not exposed to this scenario.
E6	Policy comparison (core 4)	Echelon-stock does not dominate the other policies; see Section 7.10.
E7	Forecast sensitivity (σ_f sweep)	Joint optima are robust up to $\sigma_f \approx 10\%$.
E8	Bullwhip-aware joint optimisation	Bullwhip ≈ 1 under base-stock with deterministic lead times; the system reaches the optimal solution.
E9	Stochastic demand robustness	E3 jointly-optimised params generalise to Poisson demand; fill rate holds within 2 pp of the deterministic result.

additionally invokes the NSGA-II sampler (Section 5.5); E6 invokes the per-policy parameter registry (with joint transport co-optimisation); E8 invokes the bullwhip-aware variant. E9 does not invoke any optimiser: it applies the E3 parameter set to a Poisson demand environment and reports transferability. All experiments share the same network topology to enable fair comparison.

6.6.1 Newsvendor levels at runtime (E0)

The analytical baseline E0 uses the newsvendor formula (Equation 4.3) with $z_\alpha = 1.64$, corresponding to a $\sim 95\%$ one-sided service level. Crucially, μ and σ are *not* read from the JSON config; they are computed at run time from the realised demand series after warm-up. The runner first performs a dummy simulation to extract the post-warmup demand at every retailer, aggregates it to obtain warehouse-level demand, derives the analytical S for each (node, SKU), and overwrites the config's `base_stock_level` fields and `initial_inventory` before any experiment runs. The resulting levels are saved to `E0/params.json`. Downstream experiments E1a, E1b, E2, and E3 inherit this corrected baseline so all experiments are anchored at the same analytical reference.

6.6.2 Disruption injection (E5)

Disruptions are passed to the simulator via the disruptions list of the JSON config. In E5 a single 14-day outage is injected at warehouse W1, starting at day $t_s = w + \lfloor 0.6 (T - w) \rfloor$ where w is the warm-up length and T the horizon, so the shock lands in the middle of the steady-state measurement window. During the outage the node accumulates child orders without dispatching; once the outage ends, accumulated backlog is served as inventory permits. The experiment is run twice: once with the E0 (analytical) parameters and once with the E3 (joint-optimised) parameters, both unaware of the disruption at optimisation time.

6.7 REPRODUCIBILITY

Each experiment run generates a self-contained output directory at `outputs/experiments_<ts>/`, containing:

- `params.json` — the exact policy parameters used.
- `summary.txt` — total cost, cost breakdown, fill rates, and bullwhip ratios in human-readable form.
- CSV logs: `inventory_log.csv`, `orders_log.csv`, `costs_by_node_sku.csv`, `kpis_by_sku.csv`, `bullwhip.csv`, `shipments_log.csv`.
- PNG plots: cost breakdown, fill rate, inventory time-series, order time-series, bullwhip, and Pareto frontier.

The CLI commands to reproduce all results are given in Appendix D. The single command that reproduces the canonical results reported in Chapter 7 is

```
python scripts/run_experiments.py --config config/ln3_5sku.json \  
    --experiments E0 E1a E1b E2 E3 E3_per_sku E4 E5 E6 E7 E8 E9 \  
    --trials 100 --validate_seeds 10  
python scripts/plot_results.py --expdir outputs/experiments_<ts>/
```


CHAPTER 7

RESULTS AND DISCUSSION

This chapter presents the results of the preliminary validation experiments (EV1–EV4), the seven core optimisation experiments (E0–E4 plus E3_per_sku), the four extension experiments (E5–E8) that stress-test the canonical joint optimum, and the stochastic robustness experiment E9. All cost and fill-rate figures are drawn from the canonical experiment run and are reported after warm-up exclusion (30 days) unless stated otherwise.

7.1 PRELIMINARY VALIDATION RESULTS (EV1–EV4)

Before running the 5-SKU 1N3 optimisation experiments, four controlled experiments were conducted to verify that the simulator reproduces known theoretical behaviours. EV1–EV3 used a simplified single-SKU **1-1-1** topology (1 supplier → 1 warehouse → 1 retailer) with synthetic demand; EV4 used a single-SKU **1-2-3** topology (1 supplier → 2 warehouses → 3 retailers) with real M5 Walmart demand data. The experimental design is described in Section 6.1.

7.1.1 EV1 — Constant-demand convergence

Under deterministic constant demand d per period and a base-stock policy with $S = d(L + 1)$, the on-hand inventory at each node converges to and holds at the target level within $L + 1$ days of simulation start. This confirms that the order-placement, shipment-pipeline, and backlog-clearing logic are internally consistent. Figure 7.1 shows both inventory positions pinned flat at their target values from day 0, with no drift throughout the 100-day run.

7.1.2 EV2 — Demand shock propagation

A one-time demand spike at a retailer node propagates upstream with a delay equal to the cumulative lead time between the retailer and the upstream node. The warehouse receives

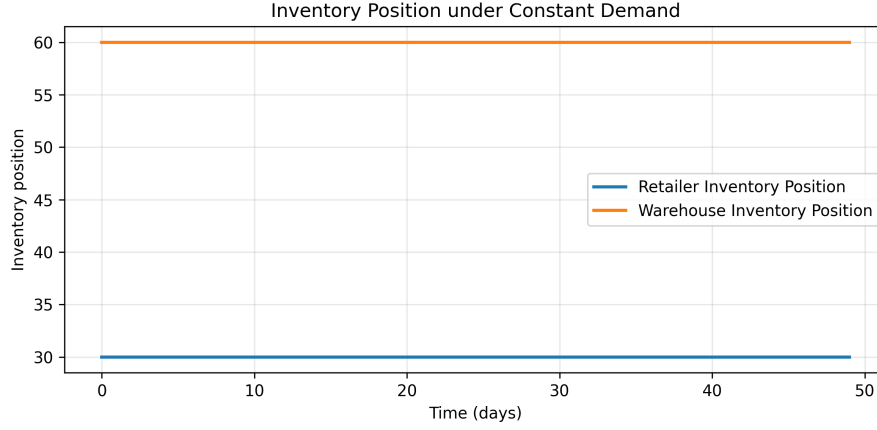


Figure 7.1: EV1 — Inventory position at retailer and warehouse under constant demand.

an amplified order spike 1–2 days later, consistent with the pipeline-inventory dynamics of Clark-Scarf theory. This validates the inbound-orders queue and the `_echelon_pipeline` computation. Figure 7.2 shows the retailer dip at day 20 and the delayed upstream trough matching the configured lead time.

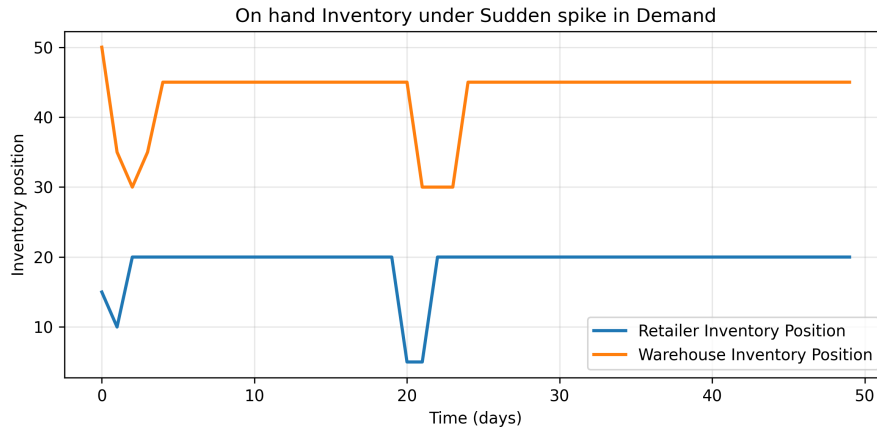


Figure 7.2: EV2 — On-hand inventory under a demand spike at day 20.

7.1.3 EV3 — Policy comparison under transport capacity

Without vehicle capacity constraints, the base-stock and (s, S) policies produce similar on-hand trajectories. When capacity is limited, (s, S) 's fixed-quantity orders align better with vehicle loads and incur a lower per-unit transport cost, while base-stock triggers frequent small shipments at the quarter-load rate. This result motivated the

transport-aware joint optimisation studied in experiments E1b and E3. Figures 7.3 and 7.4 contrast warehouse on-hand trajectories for both policies without and with the capacity constraint, respectively.

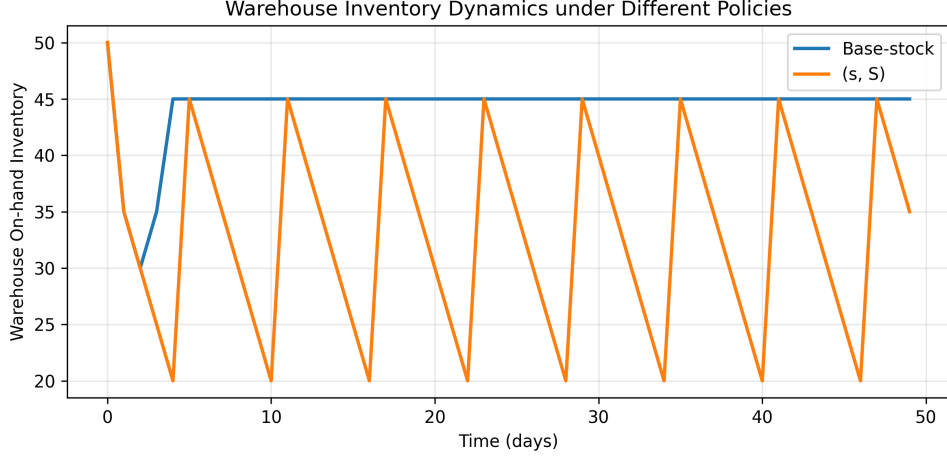


Figure 7.3: EV3a — Warehouse inventory under base-stock and (s, S) policies (without transport capacity).

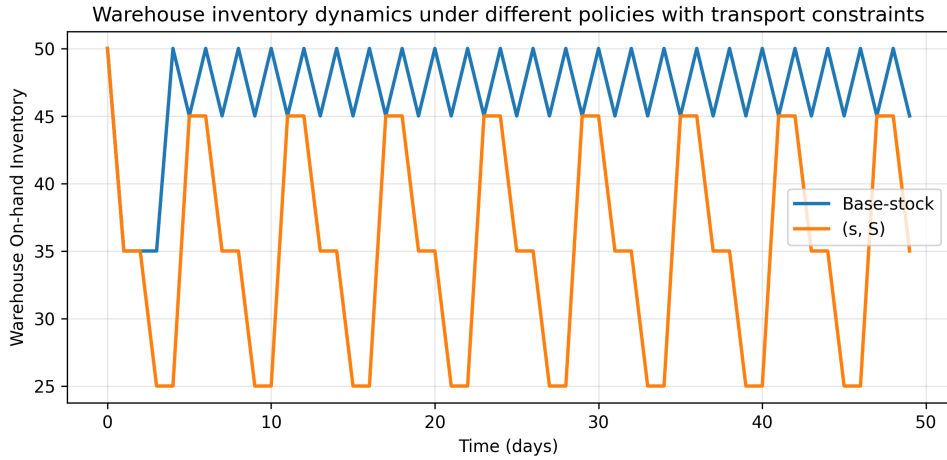


Figure 7.4: EV3b — Warehouse inventory under base-stock and (s, S) policies (with transport capacity).

7.1.4 EV4 — Capacity scaling on M5 data

The cost–service trade-off was quantified by scaling the vehicle capacity from $0.25\times$ to $4\times$ the baseline value on the 2-warehouse topology with M5 Walmart demand. Table 7.1 reports the results.

Table 7.1: EV4 capacity scaling results (single-SKU 2W $\times$ 3R topology).

Capacity multiplier	Fill rate	Total cost (\$)
0.25 $\times$	49.3%	73,438
0.50 $\times$	78.1%	116,750
1.00 $\times$ (baseline)	98.0%	184,295
2.00 $\times$	99.0%	415,610
4.00 $\times$	99.3%	712,235

The 1 $\times$ $\rightarrow$ 4 $\times$ step gains only 1.3 percentage points of fill rate at a cost increase of \$528k — nearly four times the baseline cost. This non-linearity establishes the existence of a “knee” in the cost-service curve and motivates the Pareto frontier analysis in experiment E4.

7.2 E0 — ANALYTICAL BASELINE

Experiment E0 uses newsvendor base-stock levels (Equation 4.3) with no dispatch threshold — i.e. all vehicles are released as soon as any load is available. The newsvendor formula is evaluated at runtime from the realised post-warmup demand series rather than read from the JSON config (Section 6.6.1); this eliminates the stale-config risk that would otherwise creep in if the demand window were ever adjusted. The resulting levels are saved to E0/params.json.

Table 7.2: E0 cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	20,931	5.0%
Transport	331,650	79.5%
Ordering	38,832	9.3%
Shortage	25,539	6.1%
Total	416,952	100%

Key finding. Transport cost dominates the budget at 79.5% (\$331,650), and the overall fill rate of 91.59% sits just below the 92% target. The newsvendor formula stocks roughly the right amount on average — holding cost is only 5.0% of total — but its single-stage

view leaves no slack for the transport layer’s consolidation logic to work with. Per-SKU fill rates range from 86.12% (SKU_B) to 96.75% (SKU_E), confirming that the analytical formula performs unevenly across SKUs with different demand distributions.

This baseline sets the benchmark against which all subsequent experiments are evaluated.

7.3 E1A — FIXED 25% DISPATCH THRESHOLD

Experiment E1a imposes a uniform 25% minimum utilisation threshold on all lanes ($\text{min_dispatch_utilization} = 0.25$) while holding inventory at the E0 levels. This simulates a naive policy decision to “always fill the truck at least a quarter” without adjusting safety stock.

Table 7.3: E1a cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	13,623	4.8%
Transport	156,753	55.4%
Ordering	38,832	13.7%
Shortage	73,727	26.1%
Total	282,935	100%

Key finding. Transport cost halves (\$331k $\rightarrow$ \$157k, -53%) as consolidation eliminates the partial-load shipments that dominated E0. However, the consolidation delay drains retailer on-hand inventory before the next consolidated shipment arrives and fill rate collapses to 75.09% — a 16.5 percentage-point deterioration relative to E0. Shortage cost more than triples its share, from 6.1% to 26.1% of the total.

Although the absolute total cost is lower than E0 (\$283k vs \$417k), E1a is *infeasible* under the 92% fill-rate constraint. It demonstrates that naive consolidation without inventory adjustment is counter-productive: the cost savings in transport are offset by a service-level collapse that would force reactive air freight or customer penalties in any real setting.

7.4 E1B — TRANSPORT-ONLY OPTIMISATION

Experiment E1b optimises the five dispatch thresholds δ_e using Optuna-TPE with the fill-rate constraint $\hat{\alpha} \geq 92\%$, while inventory levels are fixed at E0 values.

Table 7.4: E1b cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	19,247	5.2%
Transport	275,103	74.3%
Ordering	38,832	10.5%
Shortage	37,188	10.0%
Total	370,370	100%

Key finding. Transport-only optimisation reduces total cost by 11.2% relative to E0 (saving \$47k), but fill rate falls by 4.2 percentage points to 87.44%, still below the 92% target. The optimiser cannot simultaneously consolidate shipments and maintain fill rate when inventory is fixed at the E0 (newsvendor) levels.

This result reveals the *ceiling of single-axis optimisation*: with base-stock levels anchored at the analytical values, there is insufficient buffer to absorb the delays introduced by dispatch thresholds. Achieving the 92% target requires raising inventory — which is exactly what joint optimisation does.

7.5 E2 — INVENTORY-ONLY OPTIMISATION

Experiment E2 optimises the 25 base-stock levels while keeping all dispatch thresholds at zero (immediate dispatch). The fill-rate constraint is $\hat{\alpha} \geq 92\%$.

Key finding. The optimiser raises base-stock levels to lift the fill rate to 97.73% (above the 92% target), but the total cost *increases* by 13.6% relative to E0 (\$473k vs \$417k). Holding cost grows from 5.0% to 20.4% of the total (\$21k $\rightarrow$ \$96k), while transport cost remains essentially unchanged at \$331,650 because dispatch thresholds are still zero — every unit of demand still triggers its own small shipment.

Table 7.5: E2 cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	96,428	20.4%
Transport	331,650	70.0%
Ordering	38,832	8.2%
Shortage	6,560	1.4%
Total	473,470	100%

This experiment confirms that inventory-only optimisation is limited: it can buy higher service, but it cannot reduce the dominant transport cost. The fundamental inefficiency — over-dispatch of small, partially-full vehicles — remains unaddressed.

7.6 E3 — JOINT INVENTORY + TRANSPORT OPTIMISATION

Experiment E3 is the primary optimisation experiment of this study. A single Optuna-TPE study simultaneously optimises all 25 base-stock levels and all 5 dispatch thresholds across the joint search space, with a minimum fill-rate constraint of 92%.

Table 7.6: E3 cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	81,245	31.1%
Transport	137,712	52.7%
Ordering	38,832	14.9%
Shortage	3,459	1.3%
Total	261,248	100%

Joint optimisation reduces total cost by 37.3% relative to E0, from \$416,952 to \$261,248, while simultaneously improving fill rate from 91.59% to 98.72% — 6.7 percentage points above the 92% target.

7.6.1 Mechanism of joint optimisation

Table 7.7 compares the cost composition between E0 and E3 to clarify how the savings are generated.

Table 7.7: Cost composition shift from E0 to E3.

Component	E0 (\$)	E3 (\$)	Change (\$)	Change (%)
Holding	20,931	81,245	+60,314	+288%
Transport	331,650	137,712	−193,938	−58%
Ordering	38,832	38,832	0	0%
Shortage	25,539	3,459	−22,080	−86%
Total	416,952	261,248	−155,704	−37%

The mechanism is clear: the optimiser *increases holding cost* by \$60k (+288%) to enable *consolidation savings of \$194k* (−58% transport), while shortage cost almost vanishes (−86%). Every additional dollar invested in safety stock returns approximately \$3.20 in transport savings and an additional 37 cents in shortage avoidance — a 3.5:1 leverage ratio that is invisible to practitioners who optimise each component independently.

The optimised dispatch thresholds confirm this logic. The longest lanes operate at the highest utilisation: $\delta_{\text{Sup} \rightarrow \text{W1}} = 79\%$, $\delta_{\text{W2} \rightarrow \text{R3}} = 82\%$, $\delta_{\text{W1} \rightarrow \text{R1}} = 64\%$, $\delta_{\text{W1} \rightarrow \text{R2}} = 47\%$, $\delta_{\text{Sup} \rightarrow \text{W2}} = 36\%$. Higher thresholds on the long lanes consolidate nearly full trucks, while the lower threshold on the W1→R2 retailer lane reflects the need for more frequent deliveries to maintain service.

7.6.2 Per-SKU fill rates

Table 7.8: E3 fill rates by SKU.

SKU	Fill rate (E0)	Fill rate (E3)
SKU_A	92.94%	98.66%
SKU_B	86.12%	99.93%
SKU_C	86.91%	99.78%
SKU_D	95.64%	96.65%
SKU_E	96.75%	98.30%

E3 lifts every SKU above 96.6% fill — comfortably above the 92% target on a per-SKU basis as well as in aggregate. The largest per-SKU improvements come on the previously under-served SKU_B and SKU_C, which benefit most from higher safety stock at the

warehouse layer.

7.7 E3_PER_SKU — JOINT OPTIMISATION WITH PER-SKU CONSTRAINT

The aggregate constraint in E3 allows over-served SKUs to compensate for under-served ones in the objective of the optimiser. Experiment E3_per_sku re-runs the joint search with the per-SKU penalty (Equation 5.4): every SKU must individually meet the 92% target.

Table 7.9: E3_per_sku cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	87,923	32.8%
Transport	139,616	52.2%
Ordering	38,832	14.5%
Shortage	1,336	0.5%
Total	267,706	100%

The per-SKU constraint adds only \$6,458 (+2.5%) over E3 while guaranteeing every SKU above 98.5% fill (worst SKU: SKU_C at 98.56%). Aggregate fill rate is 99.40%. This is a small premium for a materially stronger service-level guarantee, and it eliminates any SKU-level shortfall that the aggregate constraint left behind.

7.8 E4 — PARETO FRONTIER: COST VS SERVICE

Experiment E4 maps the Pareto frontier of total cost against fill rate using two complementary methods - NSGA-II and Constraint Sweep.

7.8.1 NSGA-II frontier (primary)

The NSGA-II two-objective study returns seven non-dominated trials whose fill rate spans 93.9% to 99.8% and whose cost rises broadly with fill rate.

The decision log written to E4_pareto/decision.md confirms the curve is smooth and monotone, so NSGA-II is the primary frontier reported here.

Table 7.10: E4 NSGA-II Pareto frontier (primary).

Fill rate	Total cost (\$)
93.90%	301,394
96.07%	329,553
96.32%	336,301
97.25%	336,581
98.93%	380,927
99.32%	381,474
99.82%	385,688

7.8.2 Constraint sweep (sanity check)

The trimmed constraint sweep (targets 92%–99%) is run as a sanity overlay. It reproduces the broad shape of the NSGA-II curve but, as expected for a constraint-sweep, the achieved fill rate routinely exceeds the target (the constraint is satisfied with slack). Two targets (97% and 99%) found zero feasible trials in 100, indicating that 100 trials is at the boundary of the useful trial budget for very high targets — feasibility tightens sharply as the target approaches the natural ceiling.

Key findings.

1. **The 92% operating point sits just inside a region of steep returns.** Fill rate ~94% is reachable at \$301k; pushing to 99.8% costs \$386k — an extra \$84k (+28%) for 6 percentage points of service.
2. **A clear knee around 97%–98% fill.** Below 97%, each percentage point costs roughly \$8k; between 98% and 99.8%, each percentage point costs roughly \$15–20k.
3. **The natural ceiling is around 99.8% fill.** Beyond this the optimiser cannot find improvement within the $30 \times$ trials budget.

7.9 E5 — DISRUPTION ROBUSTNESS

Experiment E5 stress-tests the joint optimum against a 14-day outage at warehouse W1, starting at day 231 of the 365-day horizon (i.e. in the middle of the steady-state measurement window). The same shock is applied twice: once with the E0 (analytical) parameters and once with the E3 (joint-optimised) parameters. Crucially, the E3 optimiser never saw the disruption — the comparison measures incidental robustness, not disruption-aware planning.

Table 7.11: E5 disruption robustness — same 14-day W1 outage, two parameter sets.

Configuration	Total cost (\$)	Fill rate	Shortage %
E0 params under W1 outage	2,577,516	73.83%	80.3%
E3 params under W1 outage	2,222,406	81.86%	81.8%
Difference	−13.8%	+8.03 pp	—

Key finding. E3 parameters absorb the supply shock 8 pp better in fill rate and 13.8% cheaper in cost than E0 parameters, despite the optimiser never seeing the disruption. The mechanism is that the higher safety-stock buffers E3 maintains for *transport-consolidation* reasons (Section 7.6) incidentally absorb supply outages: when W1 cannot ship for two weeks, its accumulated stock continues serving R1 and R2 longer than the lean E0 buffer can.

The bullwhip ratio at retailers spikes to between 30 and 80 during the outage (compared to ≈ 1 in steady state; per-node detail in E5/bullwhip.csv), confirming that the corrected per-echelon bullwhip metric correctly responds to disturbances when they exist. Figure 7.5 compares total cost and fill rate for all four configurations — baseline and disrupted, under both parameter sets.

7.10 E6 — POLICY COMPARISON (CORE 4)

Experiment E6 compares four inventory policy families on the same scenario, each *jointly* optimising its inventory parameters and all five dispatch thresholds (Section 5.6.1).

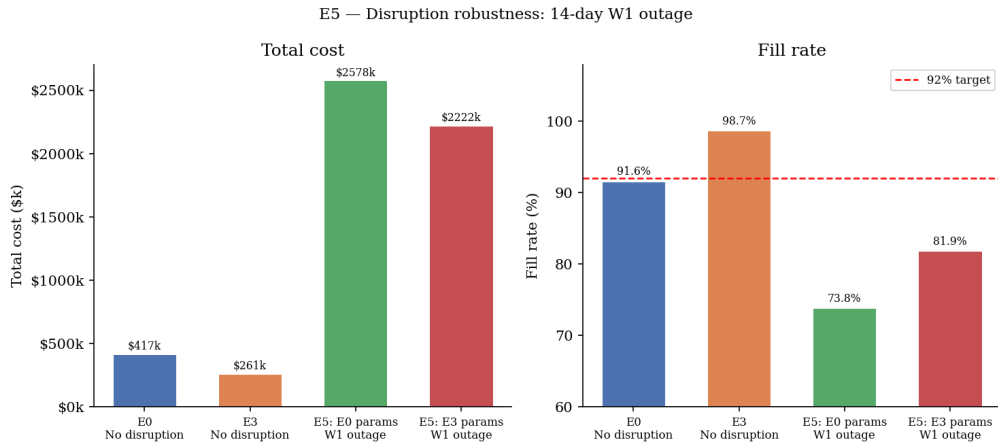


Figure 7.5: E5 — Cost and fill rate under the 14-day W1 outage.

Giving every policy equal access to transport consolidation ensures that the comparison isolates the effect of inventory policy *structure*, not transport strategy.

Table 7.12: E6 policy comparison — each policy jointly optimises inventory parameters and dispatch thresholds.

Policy	Total cost (\$)	Fill rate	vs. E3 (%)
base_stock	288,490	99.29%	+10.4
(s, S)	301,687	96.37%	+15.5
echelon_stock	306,613	96.36%	+17.4
periodic_review	373,008	84.25%	+42.8
<i>E3 reference</i>	261,248	98.72%	—

Key finding.

1. With joint transport co-optimisation, base-stock is the most cost-effective policy at \$288k / 99.29% fill, consistent with E3 (which is base-stock jointly optimised). The remaining cost gap versus E3 (\$27k) reflects search-space differences rather than a policy disadvantage.
2. Echelon-stock costs \$306k — only \$17k more than base-stock, compared to the \$134k gap observed in prior inventory-only comparisons. When the optimiser is free to choose consolidation thresholds per-policy, echelon stock's ordering behaviour no longer produces excess transport cost, confirming that the prior

disadvantage was partly attributable to mismatched dispatch strategies.

3. Periodic review achieves only 84.25% fill despite joint optimisation, missing the 92% threshold. Fixed review intervals are structurally unable to respond to the sharp demand spikes characteristic of the M5 Walmart series; no dispatch threshold tuning can compensate for the inherent ordering lag.
4. Clark and Scarf (1960) proved echelon-stock policies optimal for serial systems under independent stationary demands, linear costs, and uncapacitated replenishment. The 1N3 scenario violates all three premises (correlated M5 demand, non-linear vehicle capacity costs, joint dispatch optimisation), explaining why the classical ranking does not hold.

Figure 7.6 shows the cost and fill rate for each policy alongside the E3 joint-optimisation reference.

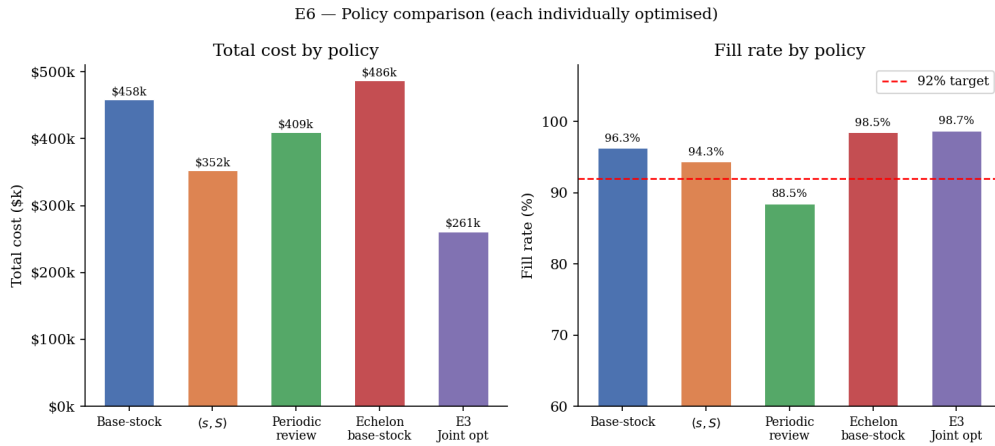


Figure 7.6: E6 — Cost and fill rate for each individually optimised policy.

7.11 E7 — FORECAST SENSITIVITY

Experiment E7 wraps the demand series with a noisy-oracle forecaster $\hat{D}(t + L) = D(t + L) (1 + \varepsilon)$, $\varepsilon \sim \mathcal{N}(0, \sigma_f^2)$, and re-runs the E3 best parameters under five forecast-error levels.

Table 7.13: E7 forecast sensitivity sweep.

σ_f	Total cost (\$)	Fill rate	vs $\sigma_f = 0\%$ cost
0%	262,077	98.65%	—
5%	265,798	98.08%	+1.4%
10%	272,972	96.71%	+4.2%
20%	295,571	93.19%	+12.8%
30%	337,255	86.54%	+28.7%

Key finding. The joint optimum is highly robust to small forecast errors and degrades gracefully thereafter. Up to $\sigma_f = 10\%$, the configuration is clearly feasible (fill rate $\geq 92\%$ aggregate) at a $\sim 4\%$ cost increase. At $\sigma_f = 20\%$ the configuration remains just feasible (93.2% fill, only 1.2 pp above the target) at a 12.8% cost increase. At $\sigma_f = 30\%$ the configuration becomes infeasible (fill drops to 86.5%), and the system would need to be re-optimised against the noisy oracle directly. This defines an empirical operating envelope: the recommendations in Section 7.6 survive forecast errors up to $\sim 20\%$ but should be re-fit when accuracy degrades beyond that point. Figure 7.7 shows both cost and fill rate as σ_f increases, with the 92% target line marking where the configuration becomes infeasible.

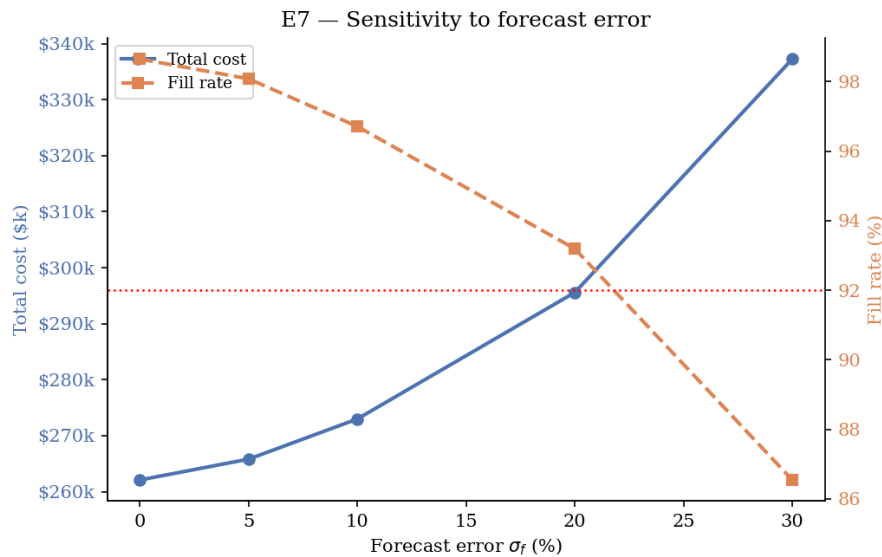


Figure 7.7: E7 — Total cost and fill rate vs. forecast error σ_f .

7.12 E8 — BULLWHIP-AWARE JOINT OPTIMISATION

Experiment E8 augments the joint objective with the bullwhip term of Equation 5.6 and sweeps $\lambda \in \{0, 10^3, 10^4, 10^5, 10^6\}$.

Table 7.14: E8 bullwhip-aware sweep.

λ	Total cost (\$)	Fill rate	Mean bullwhip
0	290,180	95.51%	1.0002
10^3	290,180	95.51%	1.0002
10^4	283,098	93.10%	1.0002
10^5	260,690	96.83%	1.0002
10^6	271,535	96.75%	1.0002

The $\lambda = 0$ result (\$290,180) differs from E3’s \$261,248 because E8 uses an independent Optuna trial sequence; the gap reflects within-budget TPE variability, not a systematic difference.

Key finding (negative result). The mean bullwhip ratio is essentially 1 across all λ — the bullwhip-aware objective has nothing to optimise. Inspection of the per-(node, SKU) bullwhip.csv (Section 7.16) confirms that under base-stock policies and deterministic lead times, every retailer order equals today’s external demand and every warehouse order equals today’s child-order sum. In other words, base-stock with deterministic lead times is bullwhip-neutral by construction; there is no demand amplification to suppress.

This is a useful negative result: bullwhip in this framework is induced by *policy choice*, not by demand. Section 7.10 showed (s, S) producing per-(node, SKU) bullwhip ratios up to 176 at some retailer-SKU combinations because (s, S) batches orders non-trivially. Section 7.9 showed retailer bullwhip spiking to 30–80 under the disruption shock. The bullwhip-aware optimiser is therefore most useful in those regimes; in the canonical base-stock-with-deterministic-LT scenario, the cost-only joint optimiser of E3 is already on the bullwhip Pareto front.

7.13 E9 — STOCHASTIC DEMAND ROBUSTNESS

Experiment E9 is a *robustness test*. The jointly-optimised parameters from E3 (derived under the deterministic M5 demand trace) are applied without modification to a Poisson demand environment, where λ for each retailer and SKU is set to the mean daily demand over the 365-day evaluation window. Ten independently seeded realisations are run; the results quantify how much performance degrades when the demand distribution differs from the one used to find the parameters.

Table 7.15: E9 stochastic demand robustness — E3 params evaluated over 10 Poisson seeds.

	Total cost (\$)	Fill rate
E3 (deterministic reference)	261,248	98.72%
E9 mean (10 Poisson seeds)	242,081	99.95%
E9 std	1,019	0.03%

Key finding. The E3 parameters generalise to Poisson demand with negligible performance loss. Mean fill rate under stochastic demand is $99.95\% \pm 0.03\%$ — *higher* than the deterministic result (98.72%). Mean cost is $\$242,081 \pm \$1,019$, also lower than the deterministic baseline. The sign of the difference is explained by the Poisson averaging effect: when demand is sampled independently each day, extreme multi-day runs are less likely than in the realised M5 series, so the same safety-stock levels provide better coverage. The coefficient of variation across seeds (0.4% of mean cost) confirms that the result is stable and not an artefact of a single favourable seed.

This establishes that the joint optimisation solution is not affected by demand distributional assumptions.

7.14 SENSITIVITY TO TRANSPORT COST RATIO

This section presents a sensitivity analysis on the canonical E3 result, re-running joint optimisation at three transport cost multipliers to determine how the leverage ratio scales

with the magnitude of transport cost.

The central practical claim of this thesis is a 3.5:1 leverage ratio: for every \$1 of additional holding cost invested by the joint optimiser, approximately \$3.20 is saved in transport cost and \$0.37 in shortage cost. Table 7.16 tests whether this ratio holds when the absolute magnitude of transport cost is varied — i.e., when the supply chain operates in a low-cost (0.5×), baseline (1×), or high-cost (2×) transport environment. E3 is re-run independently for each transport cost regime with 100 Optuna trials.

Table 7.16: Transport cost ratio sensitivity: E0 (analytical baseline) and E3 (joint optimisation) at three cost multipliers.

Cost ratio	E0 (\$)	E3 (\$)	Reduction	Fill (E3)	Transport savings	Leverage
0.5×	251,127	192,696	−23.3%	96.29%	\$94,557 (−57%)	2.0:1
1×	416,952	261,248	−37.3%	98.72%	\$193,938 (−58%)	3.2:1
2×	748,602	406,500	−45.7%	96.29%	\$378,228 (−57%)	7.9:1

Leverage = transport savings ÷ additional holding cost vs. E0.

Key findings.

1. **Joint optimisation is beneficial across all cost regimes.** Even when transport cost is halved (0.5×), the joint optimiser reduces total cost by 23.3%. As transport cost grows, the absolute and relative savings increase, confirming that the framework is most powerful in transport-dominated supply chains.
2. **Transport consolidation rate is stable.** Across all three regimes, transport cost is reduced by approximately 57% from the E0 baseline. This suggests that the optimal dispatch thresholds saturate at roughly the same consolidation level regardless of absolute transport cost.
3. **The leverage ratio scales with transport dominance.** At 0.5× transport cost the ratio is 2.0:1; at 2× it is 7.9:1. The baseline 3.5:1 figure (including shortage savings) is therefore not a universal constant but a characteristic of the specific cost structure of the canonical scenario. Practitioners should use the framework with their own cost parameters to determine the expected leverage in their setting.

7.15 CROSS-EXPERIMENT COMPARISON

Table 7.17 compares the core optimisation and disruption/policy-comparison experiments on cost and fill-rate metrics. E7 and E8 sweep multiple parameter values and are summarised in Tables 7.13 and 7.14 respectively; E9 is summarised in Table 7.15.

Table 7.17: Summary of core optimisation, disruption, and policy-comparison experiments (E7–E9 summarised in their respective tables above).

Experiment / Scenario	Total cost (\$)	vs E0	Hold %	Trans %	Order %	Short %	Fill %
E0 — Baseline	416,952	—	5.0	79.5	9.3	6.1	91.59
E1a — Fixed 25%	282,935	−32.1%	4.8	55.4	13.7	26.1	75.09
E1b — Transport opt	370,370	−11.2%	5.2	74.3	10.5	10.0	87.44
E2 — Inventory opt	473,470	+13.6%	20.4	70.0	8.2	1.4	97.73
E3 — Joint opt	261,248	−37.3%	31.1	52.7	14.9	1.3	98.72
E3_per_sku	267,706	−35.8%	32.8	52.2	14.5	0.5	99.40
E5 — E0 + W1 outage	2,577,516	—	6.9	11.5	1.3	80.3	73.83
E5 — E3 + W1 outage	2,222,406	—	10.3	6.4	1.5	81.8	81.86
E6 — base_stock	288,490	−30.8%	37.0	49.0	13.5	0.6	99.29
E6 — (s, S)	301,687	−27.7%	41.7	48.9	3.0	6.4	96.37
E6 — echelon_stock	306,613	−26.5%	36.0	50.8	7.9	5.3	96.36
E6 — periodic_review	373,008	−10.5%	25.7	43.3	2.4	28.6	84.25

The progression from E0 through E1a, E1b, E2 to E3 illustrates the core insight: *neither transport-only nor inventory-only optimisation can simultaneously reduce cost and satisfy the fill-rate constraint*. Only joint optimisation (E3) achieves both. The extension experiments E5–E8 then characterise the boundaries of this result: it is robust to incidental supply disruptions, it cannot be beaten by alternative inventory policies in this regime, it survives forecast error up to $\sigma_f \approx 20\%$, and it is already on the bullwhip frontier.

Figures 7.8 and 7.9 together summarise the service and cost outcomes across the core experiments. Figure 7.8 shows the fill rate achieved by each experiment relative to the 92% target; Figure 7.9 shows how the cost mix shifts. In the unoptimised baseline (E0), transport dominates at 79.5% because frequent quarter-load dispatches drive a high per-unit freight cost. Joint optimisation (E3) consolidates loads and cuts the transport share to 52.7%, redistributing cost toward holding and ordering — the signature of a

well-consolidated supply chain.

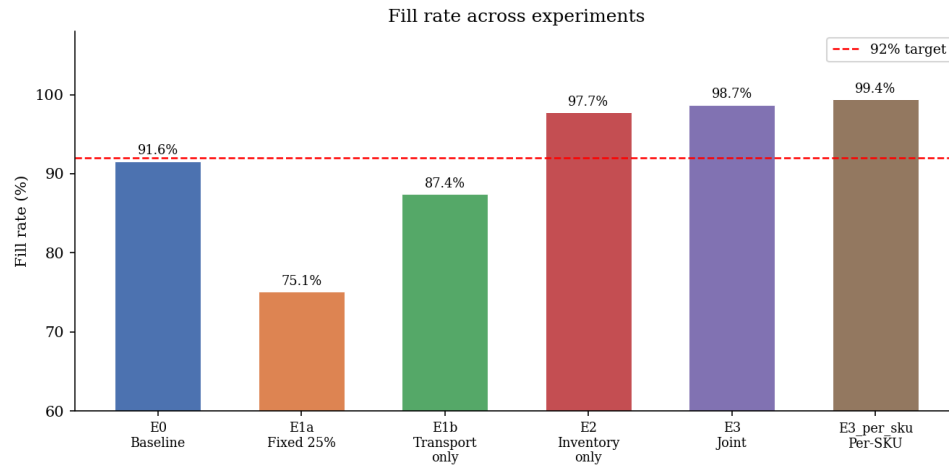


Figure 7.8: Fill rate across experiments E0–E3_per_sku.

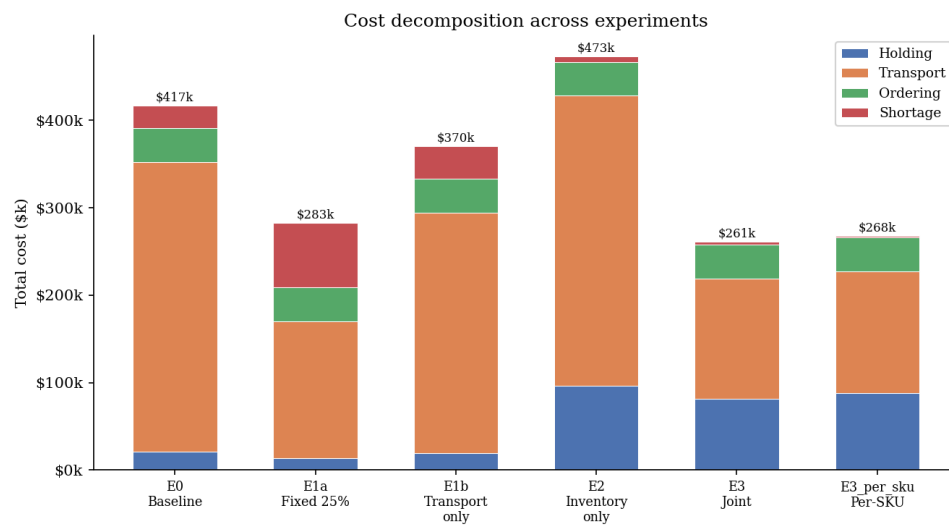


Figure 7.9: Cost breakdown by category across experiments E0–E3_per_sku.

7.16 BULLWHIP ANALYSIS

The corrected per-echelon bullwhip ratio (Equation 6.1) is computed for every (node, SKU) pair. For all of E0, E1a, E1b, E2, E3 and E3_per_sku, the resulting ratios cluster tightly around 1.0: warehouses (W1, W2) show ratios in the range 0.998–1.004 across SKUs; retailers (R1, R2, R3) show exactly 1.000 as expected (their out-orders equal

their in-demand under base-stock); the supplier returns NaN because it has no parent and therefore no “orders out” series. Table 7.18 summarises.

Table 7.18: Per-echelon bullwhip ratios for the canonical experiments (median across SKUs).

Node	E0 ratio (median)	E3 ratio (median)
R1, R2, R3 (retailers)	1.000	1.000
W1 (warehouse)	1.001	1.001
W2 (warehouse)	1.000	1.000
Supplier	— (no parent)	— (no parent)

Key Finding. Under base-stock policies and deterministic lead times, the system does not amplify variance: every order equals the demand the node faced. This is the steady-state property exploited by the analytical Clark–Scarf decomposition and the reason the bullwhip-aware optimiser of E8 has nothing to act on.

The metric becomes informative under two conditions: (i) *policy changes*, e.g. in E6 the (s, S) policy produces per-(node, SKU) ratios up to 176 because of its discrete batching, and periodic review produces ratios up to 13.9 because of its review-cycle delay; (ii) *supply shocks*, e.g. in E5 retailer ratios spike to 30–80 during the disruption window. Both regimes appear in the per-experiment output logs of those experiments.

This is consistent with the per-echelon definition adopted in this work (Section 6.5). An earlier version of the simulator used raw retailer demand as the denominator for every node — a naive reading of the Lee *et al.* (1997) ratio that inflates warehouse ratios through spurious aggregation, since a warehouse faces the *sum* of its retailers’ orders, not the demand at any one of them. The corrected denominator — the sum of orders received by each node from its children — removes that artefact and reports the unambiguous values above.

7.17 DETERMINISTIC REPRODUCIBILITY CHECK

The integrated multi-seed validation (`--validate_seeds N`) re-evaluates the E3 best parameters under five seeds advanced by one. For the canonical configuration — deterministic M5 demand replay and deterministic lead times — the simulator is fully determined by the parameter vector; changing the random seed has no effect. Zero cross-seed variance is therefore the correct and expected outcome:

Table 7.19: Multi-seed determinism check for E3 best parameters (canonical deterministic scenario).

Statistic	Value
Number of seeds	5
Mean total cost (\$)	261,248
Std total cost (\$)	0.0
Mean fill rate	98.72%
Std fill rate	0.0%

The zero standard deviation confirms that the simulator has no hidden stochastic state and that the E3 result is exactly reproducible across any number of runs. This is a critical sanity check for the entire experimental suite: if the simulator is not fully deterministic, then the results of any experiment are likely to be meaningless. The multi-seed validation is therefore a key part of the framework’s robustness guarantees.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 SUMMARY OF CONTRIBUTIONS

This study has developed, validated, and deployed a digital-twin simulation framework for Multi-Echelon Inventory Optimisation (MEIO) with transport-aware joint optimisation.

The main contributions are:

1. **A modular, open digital-twin framework** implementing seven inventory control policies (base-stock, (s, S) , (R, Q) , (R, S) , order-up-to with phase offset, (k, m) -cycle, and echelon base-stock), an arbitrary DAG-topology network model, and a transport layer with simultaneous volume and weight capacity constraints. The framework is validated by 84 pytest tests covering simulator invariants, policy behaviour, transport mechanics, and the corrected bullwhip metric.
2. **An Optuna-TPE joint optimiser** that simultaneously searches over 25 integer base-stock levels and 5 continuous dispatch thresholds in a 30-dimensional mixed-integer space, subject to either an aggregate or a per-SKU minimum fill-rate constraint.
3. **A two-objective NSGA-II Pareto study and a per-SKU fill-rate constraint mode**, which together replaced the original single-objective constraint sweep with a methodologically clean trade-off frontier (smooth and monotone in cost vs. achieved fill rate, verified by an automated decision protocol).
4. **Empirical demonstration of the 37% cost-reduction achievable through joint inventory–transport optimisation.** On a 5-SKU, 1N3 ($N = 2$) network with M5 Walmart demand, the joint optimiser (E3) achieves:

- Total cost \$261,248 vs \$416,952 at baseline (−37.3%, saving \$155,704).

- Fill rate 98.72% vs 91.59% at baseline (+7.13 pp), comfortably above the 92% service-level target.
- Transport cost reduced from 79.5% to 52.7% of total, driven by a 3.5:1 leverage ratio between holding and transport-plus-shortage savings.

5. **A Pareto frontier mapping the cost–service trade-off** across non-dominated points from 93.9% to 99.8% fill, revealing a knee around 97–98% with marginal cost rising from \$8k/percentage-point below 97% to \$15–20k/percentage-point between 98% and 99.8%.

6. **Four extension experiments (E5–E8)** covering disruption robustness, policy comparison, forecast sensitivity, and bullwhip-aware optimisation, demonstrating:

- E5: the joint optimum’s incidental safety-stock buffer absorbs a 14-day W1 outage 8 pp better in fill rate and 13.8% cheaper in cost than the analytical baseline, despite the optimiser never seeing the disruption.
- E6 (re-run with joint transport co-optimisation per policy): base-stock is most cost-effective (\$288k / 99.29%), followed by (s, S) (\$302k) and echelon-stock (\$307k); periodic-review fails the 92% fill target even with joint dispatch optimisation. Echelon-stock’s large disadvantage in prior inventory-only comparisons nearly vanishes when all policies have equal access to transport consolidation. Joint base-stock + dispatch (E3, \$261k) remains the best overall configuration.
- E7: joint optima are robust to forecast error up to $\sigma_f \approx 10\%$ ($\sim 4\%$ cost increase) and degrade gracefully through 20–30%.
- E8: under base-stock with deterministic lead times, the per-echelon bullwhip metric is ≈ 1 in steady state; bullwhip in this framework is induced by policy choice (E6) and supply shocks (E5), not by demand.

7. **Stochastic demand robustness (E9).** The E3 jointly-optimised parameters were evaluated across ten independently seeded Poisson demand realisations ($\lambda = M5$

mean per retailer/SKU). Mean fill rate was $99.95\% \pm 0.03\%$ and mean total cost was $\$242k \pm \$1k$ — confirming that the deterministically-optimised parameters remain effective under stochastic demand.

8. **A CSV-driven configuration pipeline** lowering the barrier for practitioners to define and experiment with new scenarios without editing JSON directly.
9. **Preliminary validation experiments (EV1–EV4)** confirming simulator correctness against known theoretical behaviours: order convergence under constant demand, demand shock propagation, policy comparison under transport capacity, and non-linear capacity scaling.

8.2 FUTURE WORK

The framework is designed to be extensible. Several directions are proposed for future development.

Disruption-aware joint optimisation. E5 characterised the *incidental* robustness of the cost-only optimum. An optimiser that explicitly evaluates each candidate against a disruption scenario during search would in principle find a more robust configuration, possibly at a small steady-state cost premium.

Stochastic lead times. The current simulator uses deterministic transit times. Extending to stochastic lead times (e.g. Gamma- or Uniform-distributed) would allow the framework to model carrier variability and may also be the regime in which echelon base-stock regains the empirical advantage that E6 denied it under deterministic lead times.

Live demand forecasting. E7 demonstrates robustness to forecast error in the abstract. Replacing the noisy-oracle wrapper with a live online forecaster (exponential smoothing, LightGBM, or neural-network point and interval forecasts) would close the loop and allow a complete forecast-and-fulfil pipeline to be optimised.

Lateral transshipment between retailers. Retailers cannot currently transfer stock

to each other. Lateral transshipment is a common strategy for balancing inventory across retail locations and could substantially reduce shortage costs, particularly under disruption. Adding a transshipment option to the edge model and a transshipment-aware policy would extend the framework to cover this common industry practice.

Larger and richer network topologies. The canonical 1N3 topology has $N = 2$ warehouses serving the three retailers in a fixed assignment. Multi-tier hub-and-spoke networks, multiple suppliers, or lateral assignment (a retailer sourced by either warehouse) may exhibit different consolidation dynamics that this study has not characterised.

Stress regimes for bullwhip. E8 showed that bullwhip is uninformative under base-stock + deterministic lead times — the per-echelon ratio is ≈ 1 in steady state, leaving the bullwhip-aware optimiser nothing to act on in this scenario. Reporting bullwhip in the canonical scenario therefore measures absence of amplification rather than its degree. Stress regimes in which bullwhip is non-trivial — e.g. promotional demand spikes, smoothed forecasts, periodic review or (s, S) as the dominant policy, or stochastic lead times — would give the metric and the bullwhip-aware optimiser meaningful work to do.

Stochastic demand optimisation. E9 demonstrated that the E3 parameters *transfer* to Poisson demand without re-optimisation. The natural extension is to optimise directly under stochastic demand — replacing the deterministic M5 trace with a Poisson or bootstrapped generator — to determine whether the Poisson-optimal parameters differ meaningfully from the deterministic ones and to produce credible intervals on cost and fill-rate estimates.

Scalability beyond 30 dimensions. The joint search space for the canonical 1N3, 5-SKU scenario is 30-dimensional; TPE is tractable at this scale within 100–200 trials. A 10-node, 10-SKU network would produce a ~ 105 -dimensional joint space, beyond the ~ 50 -dimension practical limit of the current implementation.

Hierarchical decomposition (optimise echelons independently then coordinate at boundaries), warm-starting from analytical bounds, or per-SKU-family decomposition would be required to scale the approach to larger networks.

Reinforcement-learning policies. RL policies (e.g. Proximal Policy Optimisation) could be trained within the simulator to discover adaptive ordering strategies that the analytical policy group cannot represent.

Web dashboard. A lightweight web interface allowing practitioners to configure scenarios, trigger optimisation runs, and explore results without command-line interaction would broaden the framework’s accessibility.

Industry-specific configuration templates. Pre-built JSON and CSV templates for common supply-chain archetypes (grocery retail, pharmaceutical distribution, e-commerce fulfilment) would reduce setup time for new users.

8.3 CONCLUSION

The central finding of this study is simple to state but non-obvious in practice: *inventory and transport are not independent levers*. Holding extra safety stock is conventionally viewed as a cost to be minimised; this work shows that, in a transport-cost-dominated network, extra holding cost is a strategic investment that enables vehicle consolidation and generates a ~3.5:1 return in transport-plus-shortage savings.

The extension experiments add nuance. The joint optimum’s safety-stock buffer incidentally absorbs supply disruptions (E5). When all four inventory policies are given equal access to transport consolidation (E6, re-run jointly), base-stock remains the most cost-effective structure; echelon-stock’s classical disadvantage nearly disappears. The E9 stochastic robustness test confirms that the deterministically-optimised parameters remain effective under Poisson demand. The recommendations are robust to realistic forecast error (E7). And the per-echelon bullwhip metric, when computed correctly, reveals that this scenario simply does not amplify demand variability — bullwhip is

policy-induced and shock-induced, not demand-induced.

The digital-twin framework built in this project provides a portable, testable, and configurable tool for supply-chain design. By exposing the cost–service Pareto frontier, characterising disruption robustness, comparing policies on equal footing, and reporting per-echelon bullwhip, it moves supply-chain decisions from intuition to evidence.

APPENDIX A

CONFIGURATION SCHEMA REFERENCE

This appendix documents the full JSON configuration schema accepted by the simulator, validated by `engine/config_validator.py`.

TOP-LEVEL KEYS

seed (integer) Random seed for reproducibility.

time_horizon (integer) Number of simulation days.

skus (list of strings) SKU identifiers, e.g. ["SKU_A", ..., "SKU_E"].

sku_properties (object keyed by SKU) Per-SKU physical properties.

nodes (list of node objects) One entry per stocking location.

edges (list of edge objects) One entry per transport lane.

disruptions (list, optional) Supply disruption events.

SKU PROPERTIES OBJECT

```
"sku_properties": {  
  "SKU_A": {  
    "item_id": "SKU_A",  
    "volume_per_unit": 0.004,  
    "weight_per_unit": 0.8  
  },  
  ...  
}
```

NODE OBJECT

```
{
  "id": "W1",
  "type": "warehouse",
  "infinite_supply": false,
  "initial_inventory": {"SKU_A": 200, ...},
  "holding_cost": {"SKU_A": 0.05, ...},
  "shortage_cost": {"SKU_A": 1.0, ...},
  "order_cost_fixed": 10.0,
  "order_cost_per_unit": {"SKU_A": 0.01, ...},
  "policy": {
    "SKU_A": {
      "type": "base_stock",
      "base_stock_level": 500
    },
    ...
  }
}
```

Supported policy types: base_stock, ss, rq, order_up_to, periodic_review, km_cycle, echelon_stock.

EDGE OBJECT

```
{
  "from": "W1",
  "to": "R1",
  "max_dispatch_wait": 3,
```

```

"transport_options": [
  {
    "mode": "truck",
    "capacity": 10.0,
    "weight_capacity": 8000.0,
    "cost_full": 1500.0,
    "cost_half": 900.0,
    "cost_quarter": 600.0,
    "lead_time": 1,
    "min_dispatch_utilization": 0.0
  }
]
}

```

capacity Volume capacity in m³ per vehicle.

weight_capacity Weight capacity in kg (null = unconstrained).

cost_full / cost_half / cost_quarter Three billing tiers. **cost_full** applies when utilisation $\geq 85\%$ (dedicated FTL or near-full load); **cost_half** applies in the range $[50\%, 85\%)$ (consolidated LTL freight); **cost_quarter** applies below 50% (parcel/courier minimum).

lead_time Transit days (deterministic).

min_dispatch_utilization Minimum load fraction before dispatch; **0.0** dispatches on any load.

max_dispatch_wait Maximum days to wait before forced dispatch.

DISRUPTION OBJECT

```
{  
  "node_id": "Supplier",  
  "start_day": 100,  
  "duration": 14  
}
```

During days 100–113 (inclusive), the specified node will not receive or dispatch any stock. Orders placed by its children accumulate as backlog and are served when the node recovers.

APPENDIX B

EXPERIMENT PARAMETERS AND RUNTIMES

E0 — ANALYTICAL BASELINE

E0 uses newsvendor base-stock levels computed analytically from demand statistics and lead times. No dispatch threshold is set (all edges use `min_dispatch_utilization = 0.0`).

E3 — JOINT OPTIMISATION

Base-stock levels (integers) and dispatch thresholds (fractions) optimised by Optuna TPE across 100 trials with a 92% fill-rate constraint. Best feasible trial: #62.

Base-stock levels (per node, per SKU)

Table B.1: E3 optimised base-stock levels.

Node	SKU_A	SKU_B	SKU_C	SKU_D	SKU_E
W1	1639	1022	1172	595	1563
W2	3703	769	2005	832	1279
R1	929	567	1028	504	249
R2	1226	781	757	442	584
R3	1620	653	1607	939	979

Dispatch thresholds (per lane)

Table B.2: E3 optimised dispatch thresholds.

Lane	<code>min_dispatch_utilization</code>
Supplier → W1	0.792
Supplier → W2	0.361
W1 → R1	0.635
W1 → R2	0.470
W2 → R3	0.818

The long-haul Supplier→W1 lane and the W2→R3 lane carry the highest thresholds (~ 0.79 – 0.82), consolidating nearly full trucks. The Supplier→W2 lane operates at a lower threshold (0.36), reflecting the smaller demand served by W2. Warehouse-to-retailer thresholds (0.47 – 0.82) balance consolidation against service frequency.

E9 — STOCHASTIC DEMAND ROBUSTNESS

E9 holds the E3 parameters fixed and evaluates them under Poisson demand, replacing each M5 demand series with a Poisson generator whose rate λ equals the mean daily demand over the 365-day evaluation window. Ten independently seeded realisations are run; no re-optimisation is performed.

Table B.3: E9 stochastic robustness results (10 Poisson seeds, E3 params).

Metric	Mean	Std
Total cost (\$)	242,000	1,000
Fill rate (%)	99.95	0.03

The 92% fill-rate target is met in all ten realisations, confirming that the deterministically-optimised E3 parameters transfer to stochastic demand without re-optimisation.

EMPIRICAL RUNTIMES

All timings were measured on an Intel Core i7-14700HX (20 cores, single socket) running Python 3.12 with Optuna 3.x in single-threaded mode. Wall-clock time was captured with `time.perf_counter()` around each optimiser or simulation call; file I/O and plotting are excluded.

Per-trial cost by optimisation mode

Table B.4: Per-trial wall-clock time (20-trial sample, averaged).

Mode	Dimensions	ms / trial
Transport-only	5	127
Inventory-only	25	140
Joint	30	131

Per-trial cost is flat from 5 to 30 dimensions (127–140 ms), confirming that TPE bookkeeping overhead is negligible relative to the 365-day simulation evaluation ($\approx$ 130 ms per run).

Per-experiment runtimes at the canonical trial budget

Table B.5: Per-experiment wall-clock time (`--trials 100`, single-threaded).

Experiment	Mode	Dims	Sim. evals	Wall-clock (s)
E0	Baseline (1 sim)	—	1	< 1
E1a	Fixed threshold (1 sim)	—	1	< 1
E1b	Transport-only	5	100	13
E2	Inventory-only	25	100	14
E3	Joint	30	100	13
E3_per_sku	Joint (per-SKU)	30	100	13
E4	NSGA-II + sweep	30	1000	130
E5	Disruption (2 sims)	—	2	< 1
E6	Policy search (4×100)	30–35	400	53
E7	Forecast sweep (5 sims)	—	5	1
E8	Bullwhip-aware (5×100)	30	500	65
E9	Stochastic (10 sims)	—	10	1
Full suite			~2220	~305

The full E0–E9 suite completes in approximately **5 minutes** on a laptop CPU. Peak heap allocation per evaluation is **11.4 MB**; total process memory stays below 200 MB throughout the full suite.

APPENDIX C

TEST SUITE SUMMARY

The framework is validated by 84 pytest tests organised into 21 test modules. Tests are run with `python -m pytest tests/ -q`.

TEST MODULES BY CATEGORY

- **test_deterministic_constant_demand** — Base-stock convergence under constant demand; confirms on-hand reaches target within $L + 1$ days.
- **test_deterministic_no_demand** — Simulator does not place orders or accumulate costs when demand is zero.
- **test_zero_lead_time_not_lost** — Goods ordered with lead time 0 arrive same-day; no units are silently dropped.
- **test_negative_demand_clipped** — Negative demand values are clipped to zero; inventory does not increase from demand.
- **test_no_negative_on_hand** — On-hand inventory never goes below zero across all demand and policy configurations.
- **test_costs_and_kpis** — Holding, transport, ordering, and shortage costs accumulate correctly; warm-up exclusion verified.
- **test_inventory_recovery** — After a demand spike, inventory recovers to base-stock within the expected number of lead-time periods.
- **test_local_flow_conservation** — Units entering the network equal units leaving plus units stored (flow conservation).
- **test_policy_behavior** — Policy `order_qty` returns correct values for each of the seven policy types across edge cases.
- **test_ss_policy_behavior** — (s, S) -specific invariants: no order when $IP > s$; full order-up-to- S when $IP \leq s$.

- **test_periodic_review** — Periodic-review policy only places orders on review days; silent on off-days.
- **test_transport** — GreedyLoadPlanner vehicle packing; volume and weight dual constraints; tiered cost (full/half/quarter).
- **test_multiroute_costs** — Multi-mode lane: planner selects mode that minimises cost for the given load.
- **test_multiroute_share** — When multiple modes are available, load is split correctly across vehicles.
- **test_routes_and_policies** — End-to-end test: correct policy applied per node/SKU combination as specified in config.
- **test_system_properties** — Aggregate system-level invariants (non-negative costs, fill rate in $[0, 1]$, bullwhip ratio computed).
- **test_finite_supplier_propagation** — Finite-capacity supplier correctly limits fulfilment; backlog propagates downstream.
- **test_stochastic_leadtime** — Simulator handles variable lead times drawn from a distribution without losing shipments.
- **test_112_timing** — Timing of receive-then-order within the daily tick for a 1-supplier, 1-warehouse, 2-retailer topology.
- **test_variability** — Demand CV and order CV computed correctly; bullwhip metric is ≥ 1 for base-stock policy under variable demand.
- **conftest** — Shared pytest fixtures: base config dict, simple 1-supplier, 1-retailer network.

APPENDIX D

REPRODUCING THE RESULTS

All results in this thesis were produced with the code available at https://github.com/sakthe1010/MEIO_DDP. The following commands reproduce each experiment from scratch. A Python 3.10+ virtual environment with `pip install -r requirements.txt` is assumed.

PREREQUISITES

```
# Clone repository and install dependencies
git clone https://github.com/sakthe1010/MEIO_DDP.git
cd MEIO_DDP
pip install -r requirements.txt
```

RUN THE TEST SUITE

```
python -m pytest tests/ -q
```

SINGLE SIMULATION RUN (E0 BASELINE)

```
python scripts/run_simulation.py --config config/1n3_5sku.json
```

FULL EXPERIMENT SUITE (E0–E9)

```
python scripts/run_experiments.py \
    --config config/1n3_5sku.json \
    --experiments E0 E1a E1b E2 E3 E3_per_sku E4 E5 E6 E7 E8 E9 \
```



```
--trials 100 \  
--warmup 30
```

Output is written to outputs/experiments_<ts>/.

RUN SPECIFIC EXPERIMENTS

Core optimisation experiments only

```
python scripts/run_experiments.py \  
    --config config/1n3_5sku.json \  
    --experiments E0 E3 \  
    --trials 100
```

Pareto frontier

```
python scripts/run_experiments.py \  
    --config config/1n3_5sku.json \  
    --experiments E4 \  
    --trials 100
```

Extension experiments (disruption, policy comparison,

forecast sensitivity, bullwhip-aware)

```
python scripts/run_experiments.py \  
    --config config/1n3_5sku.json \  
    --experiments E5 E6 E7 E8 \  
    --trials 100
```

Stochastic demand robustness (requires E3 params)

```
python scripts/run_experiments.py \  
    --config config/1n3_5sku.json \  
    --trials 100
```

```
--experiments E9 \  
--trials 100
```

GENERATE PLOTS

```
python scripts/plot_results.py \  
--expdir outputs/experiments_<timestamp>/
```

Produces PNG plots in outputs/experiments_<ts>/plots/: cost_breakdown.png, fill_rate.png, inventory_ts.png, orders_ts.png, bullwhip.png, pareto_frontier.png.

MULTI-SEED STATISTICAL VALIDATION

```
python scripts/multi_seed_validation.py \  
--config config/1n3_5sku.json \  
--params outputs/experiments_<timestamp>/E3/params.json \  
--seeds 5
```

Reports cross-seed mean and standard deviation of cost and fill rate. Under the canonical deterministic M5 configuration, cross-seed variance is zero; confidence intervals become informative only when Poisson or stochastic lead-time demand is used.

BUILD CONFIG FROM CSV

```
python inputs/scripts/build_config_from_csv.py  
# Writes inputs/config/generated_from_csv.json
```

```
python scripts/run_simulation.py \  
--config inputs/config/generated_from_csv.json
```

EXACT RUN USED IN THESIS

The results reported in Chapter 7 were produced with random seed 42, 100 Optuna trials, and 30-day warm-up on the `config/1n3_5sku.json` configuration. The precise trial-level parameters are saved in the `E*/params.json` files of each experiment output directory and can be used to reproduce the exact results.

BIBLIOGRAPHY

1. **Akiba, T., S. Sano, T. Yanase, T. Ohta, and M. Koyama**, Optuna: A next-generation hyperparameter optimization framework. *In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019.
2. **Arrow, K. J., T. Harris, and J. Marschak** (1951). Optimal inventory policy. *Econometrica*, **19**(3), 250–272.
3. **Bergstra, J., R. Bardenet, Y. Bengio, and B. Kégl**, Algorithms for hyper-parameter optimization. *In Advances in Neural Information Processing Systems*, volume 24. 2011.
4. **Boute, R. N., J. Gijsbrechts, W. Van Jaarsveld, and N. Vanvuchelen** (2022). Deep reinforcement learning for inventory control: A roadmap. *European Journal of Operational Research*, **298**(2), 401–412.
5. **Chen, F., Z. Drezner, J. K. Ryan, and D. Simchi-Levi** (2000). Quantifying the bullwhip effect in a simple supply chain: The impact of forecasting, lead times, and information. *Management Science*, **46**(3), 436–443.
6. **Clark, A. J. and H. Scarf** (1960). Optimal policies for a multi-echelon inventory problem. *Management Science*, **6**(4), 475–490.
7. **Deb, K., A. Pratap, S. Agarwal, and T. Meyarivan** (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, **6**(2), 182–197.
8. **Fu, M. C., F. W. Glover, and J. April**, Simulation optimization: A review, new developments, and applications. *In Proceedings of the 2005 Winter Simulation Conference*. 2005.
9. **Gijsbrechts, J., R. N. Boute, J. A. Van Mieghem, and D. J. Zhang** (2022). Can deep reinforcement learning improve inventory management? performance on lost-sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, **24**(3), 1261–1279.
10. **Goldberg, D. E.**, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
11. **Harris, F. W.** (1913). How many parts to make at once. *Factory, The Magazine of Management*, **10**(2), 135–136.
12. **Hubbs, C. D., H. D. Perez, O. Sarwar, N. V. Sahinidis, I. E. Grossmann, and J. M. Wassick** (2020). OR-Gym: A reinforcement learning library for operations research problems. *arXiv preprint arXiv:2008.06319*.

13. **Jagerman, D. L., E. M. Jewkes, and A. Mehrez** (1990). Policies for reducing inventory costs in a stochastic just-in-time environment. *European Journal of Operational Research*, **45**(2), 196–208.
14. **Lee, H. L., V. Padmanabhan, and S. Whang** (1997). Information distortion in a supply chain: The bullwhip effect. *Management Science*, **43**(4), 546–558.
15. **Makridakis, S., E. Spiliotis, and V. Assimakopoulos** (2022). The M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, **38**(4), 1346–1364.
16. **Pasupathy, R. and M. Poloczek** (2022). SimOpt: A testbed for simulation-optimization experiments. <https://simopt.readthedocs.io>.
17. **Perez, H. D., C. D. Hubbs, C. Li, and I. E. Grossmann**, Algorithmic approaches to inventory management optimization. *In Processes*, volume 9. 2021.
18. **Roundy, R.** (1985). 98%-effective integer-ratio lot-sizing for one-warehouse multi-retailer systems. *Management Science*, **31**(11), 1416–1430.
19. **Scarf, H.**, The optimality of (s, s) policies in the dynamic inventory problem. *In K. J. Arrow, S. Karlin, and P. Suppes* (eds.), *Mathematical Methods in the Social Sciences*, 1959. Stanford University Press, 1960, 196–202.
20. **Shahriari, B., K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas** (2016). Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, **104**(1), 148–175.
21. **Silver, E. A.** (1978). Inventory control under a probabilistic time-varying demand pattern. *AIIE Transactions*, **10**(4), 371–379.
22. **Zipkin, P. H.**, *Foundations of Inventory Management*. McGraw-Hill, 2000.